

摘要

当前大语言模型正从云端服务器走向本地设备、边缘计算平台和嵌入式系统，但模型权重规模持续增长、长上下文推理普及以及多会话服务需求提升，使物理内存容量和存储带宽成为端侧部署的主要瓶颈。在 `llama.cpp` 中，原生 `mmap` 机制虽然能够实现按需加载，但在内存受限场景下容易产生同步缺页、页缓存抖动和回收不可控等问题；同时，`repack` 会引入额外匿名权重副本，连续预分配的 KV Cache 也会在长上下文和 `idle/resume` 场景中长期占据物理页。因此，仅依赖操作系统被动调页难以满足内存受限环境下的高效推理需求。

针对上述问题，本项目基于 `llama.cpp` 设计并实现了一套运行时内存优化系统，从权重加载、专家激活和 KV Cache 增长三个环节进行协同优化：（1）针对 `mmap` 与 `repack` 导致的冗余驻留问题，系统关闭 `repack`，并将权重访问转为应用层可控的匿名缓冲，减少文件页与匿名副本的重复占用；（2）针对不同模型结构设计权重流式替换机制，MoE 模型利用专家稀疏激活特性，通过 CLG 跨层预测、`0_DIRECT` 并行 I/O 和 MoE-Buffer 实现专家级动态换入换出，Dense 模型则通过 Flex 稠密层匿名环形缓冲实现层粒度替换，在极端内存约束下避免 OOM；（3）针对 KV Cache 随上下文增长和 `idle session` 长期驻留的问题，系统将逻辑上下文状态与物理驻留状态解耦，以 block 为粒度维护 `resident`、`swapped`、`resume-protected` 等状态，对 `active request` 不可见的 `idle KV block` 执行 `backing store` 写入与 `madvise(MADV_DONTNEED)` 物理页释放。

系统同时引入自适应内存预算控制，根据运行时内存压力动态调整权重驻留范围、预取强度和驱逐策略。对于 `session` 恢复场景，系统通过 `swap-in`、`resume-aware prefetch`、`defer swap-out` 和 `fast maintenance` 降低恢复首字延迟与 `active decode` 热路径开销，从而在不截断上下文、不改变 `attention` 语义、不降低模型精度的前提下降低运行时 RSS。

实验结果表明，本系统能够在保持推理正确性的同时显著降低物理内存占用。关闭 `repack` 后，Dense 模型 RSS 可降低约 3.2GB，MoE 模型 RSS 可降低约 2.1GB，并能在 2GB `cgroup` 限制下稳定运行。KV Cache 回收模块在 `ShareGPT-backed synthetic trace replay` 中，`ctx4096` 场景下降低约 611.883 MiB RSS，`ctx8192` 场景下降低约 1619.195 MiB RSS。组合测试进一步表明，权重侧优化与 KV Cache 回收作用于不同内存来源，收益可以叠加，能够在显著压缩 RSS 的同时保持接近原生路径的 `active decode` 吞吐。

综上，本项目针对端侧大语言模型推理中的权重冗余驻留、被动缺页、KV Cache 膨胀以及权重 I/O 与 KV Cache I/O 竞争等问题，构建了面向内存受限系统的运行时协同优化框架。该系统综合利用模型访存规律、虚拟内存机制、异步 I/O 和模型结构特征，为本地与边缘环境中的大语言模型高效部署提供了一种可实现、可验证、可扩展的系统级方案。

关键词： 大语言模型推理；`llama.cpp`；权重流式替换；KV Cache；异步预取；边缘计算；操作系统优化

表 1: 赛题目标完成情况

目标编号	完成情况	说明
目标 1	全部完成	完成对 LLM 推理过程的访存分析，揭示模型权重的逐层顺序访问特征与 KV Cache 随上下文增长的线性增长规律，并验证 MoE 模型中专家激活的稀疏性与局部性特征。
目标 2	全部完成	实现针对模型权重与 KV Cache 的运行时内存管理机制，通过按需加载、动态驱逐与 KV Cache 物理页回收，在保证推理正确性的前提下降低运行时物理内存占用。
目标 3	全部完成	设计并实现异步预取调度器，构建独立后台加载线程池，实现计算与 I/O 的重叠，并设计自适应窗口调优机制。
总计	全部完成	初赛阶段完成 LLM 推理内存优化系统的核心逻辑设计与实现，并完成内存节省和推理性能验证。

目录

1	项目介绍	6
1.1	项目背景及意义	6
1.1.1	大语言模型发展迅速，但内存资源成为部署瓶颈	6
1.1.2	LLM 权重访存模式高度可预测，为系统优化提供空间	6
1.1.3	KV Cache 随上下文动态增长，需要细粒度运行时回收机制	6
1.2	项目目标和分析	7
1.3	项目实现内容	7
1.4	项目完成情况	8
1.5	项目开发计划	9
1.6	项目分工	10
2	现有研究调研概况	11
2.1	LLM 推理的访存行为特征	11
2.2	推理框架	12
2.2.1	llama.cpp 推理框架	12
2.2.2	vLLM 推理框架	13
2.3	内存受限场景下的优化方案	14
2.3.1	权重 I/O 优化：模型参数的存储与访问	14
2.3.2	KV Cache I/O 优化：运行时状态管理	14
2.4	边缘端部署的特殊挑战	15
2.5	现有方案的不足与本项目切入点	16
3	系统总体设计	17
3.1	设计理念	17
3.2	系统架构	18
3.3	系统核心组成	20
3.3.1	权重管理模块	20
3.3.2	KV Cache 管理模块	20
3.3.3	统一调度与执行机制	21
4	模块设计与实现	21
4.1	双钩子接口	21
4.1.1	node_callback	22
4.1.2	weight_stream_callback	23
4.2	层索引与滑动窗口	26
4.2.1	层索引构建	26
4.2.2	跨层共享页检测	27

4.2.3	滑动窗口定义与语义	27
4.2.4	窗口保护谓词	28
4.2.5	窗口推进调度	28
4.2.6	madvise 预取与回收机制	30
4.2.7	窗口 worker 执行模型	31
4.3	Dense 权重环形缓冲	32
4.3.1	槽位分配与命中逻辑	33
4.3.2	LRU 驱逐策略	34
4.3.3	slot 重绑定逻辑	35
4.3.4	执行语义	35
4.3.5	内存释放行为	35
4.3.6	Clock 更新机制	36
4.3.7	机制总结	36
4.4	MoE 专家权重流式加载	36
4.4.1	CLG 预测与专家选择	37
4.4.2	MoE-Buffer 三态管理与指针重定向	40
4.4.3	O_DIRECT 对齐并行读	41
4.4.4	热专家保护策略	42
4.4.5	自适应预算与 repack 控制	43
4.4.6	机制总结	43
4.5	KV Cache 运行时回收模块	44
4.5.1	Lazy tail 与 lazy clear: 未使用 tail 区域的回收	45
4.5.2	Paged row index 与 idle KV 退出 active 窗口	45
4.5.3	Swap-out 与 madvise 物理页释放	46
4.5.4	Resume swap-in 与 prefetch	48
4.5.5	Fast maintenance 策略	50
4.5.6	mincore resident page 诊断	51
4.6	正确性保证	53
4.6.1	只读 mmap 与数据不变性	53
4.6.2	跨层共享页保护	53
4.6.3	回收前二次检查	54
4.6.4	窗口状态的原子更新	54
4.6.5	KV Cache 的先保存、再释放原则	54
4.7	配置参数	54
4.8	与 llama.cpp 原有功能的兼容性	56
4.9	主要改动文件清单	57
4.10	核心函数说明	57

5	系统测试	59
5.1	测试环境	59
5.2	权重管理模块测试结果	60
5.2.1	内存占用与repack优化收益	60
5.2.2	Dense模型路径：滑动窗口权重驻留机制	60
5.2.3	MoE模型路径：缓冲区工作边界与buffer有效性	61
5.2.4	极端内存约束鲁棒性	63
5.3	KV Cache 管理模块测试结果	64
5.3.1	Fast maintenance 消融结果	64
5.3.2	ctx4096 ShareGPT-backed trace replay 结果	65
5.3.3	ctx8192 ShareGPT-backed trace replay 结果	65
5.3.4	mincore resident page 诊断结果	66
5.4	KV Cache 与权重侧优化的叠加效果	67
6	现阶段存在的问题	67
6.1	受限内存场景下的性能瓶颈	68
6.2	跨层共享页的识别	68
6.3	当前主要验证 CPU KV path	68
6.4	prefetch 仍是 driver 协作式	68
7	大语言模型使用说明	68

1 项目介绍

本小节将从项目背景开始，介绍赛题初衷，接着再结合赛题目标，对项目所要进行的主要工作进行分析，最后介绍项目实现的成果。

1.1 项目背景及意义

1.1.1 大语言模型发展迅速，但内存资源成为部署瓶颈

大语言模型（Large Language Model, LLM）自问世以来，经历了从百亿参数到万亿参数的跨越式发展。以 Llama 系列模型为例，Llama-70B 拥有约 700 亿参数，在 4-bit 量化下仍需约 36.2GB 存储空间，而 Llama-3.1-405B 的模型权重更是高达数百 GB。除模型参数外，推理过程中为缓存历史对话内容而不断增长的键值缓存（Key-Value Cache, KV Cache）进一步加剧了内存压力。当上下文长度达到 8k 时，KV Cache 的内存占用可达 10GB 以上。这使得大语言模型在部署时对物理内存容量提出了极高要求。

然而，嵌入式设备、边缘计算平台等资源受限场景普遍存在内存容量小、存储带宽有限的特点，难以满足大语言模型全量加载运行的需求。传统的“全部加载”策略因物理内存不足而失效，而基于内存映射文件（memory map, mmap）的按需加载方案虽能运行，却因同步输入输出（I/O）阻塞导致推理吞吐量急剧下降。因此，如何在有限内存条件下高效运行大语言模型推理，已成为当前系统级优化领域亟待解决的关键问题。

1.1.2 LLM 权重访存模式高度可预测，为系统优化提供空间

大语言模型推理过程具有高度可预测的权重访存行为特征。在自回归生成阶段，每个 Token 的生成均需依次经过模型的所有解码层，且访问顺序严格为 Layer 0 $\rightarrow$ Layer 1 $\rightarrow \dots \rightarrow$ Layer N，无随机跳转。每个 Token 重复相同的层访问序列，且 Prefill 阶段与 Generation 阶段的访存行为一致。

这种“逐层顺序访问”的特性为虚拟内存、数据预取、滑动窗口等系统优化技术提供了天然的施展空间。访问顺序可预测意味着系统能够准确预知未来若干步需要哪些数据，从而提前发起加载请求，有效隐藏 I/O 延迟。

1.1.3 KV Cache 随上下文动态增长，需要细粒度运行时回收机制

除静态模型权重外，KV Cache 也是大语言模型推理阶段的重要内存开销，其规模会随着上下文长度和生成 Token 数量近似线性增长，在长上下文、多轮对话和批量推理场景下容易迅速占据大量物理内存。

与模型权重不同，KV Cache 属于运行时动态产生的数据，其生命周期与 Token 序列、注意力窗口和推理阶段密切相关，系统可以结合推理进度和缓存状态，对其进行按需回收与再激活，从而缓解长上下文推理中的内存膨胀问题。

1.2 项目目标和分析

项目的实现目标包括如下三个方面：

1. 目标 1：分析 LLM 推理过程的访存行为

结合大语言模型推理引擎 `llama.cpp` 的执行流程，深入分析推理过程中模型参数的加载顺序、访问频率和内存占用模式。同时，分析 KV Cache 随生成长度增长的访问特征与内存占用规律，以及专家激活在 MoE 架构模型中的稀疏性特征。通过细粒度访存行为分析，为后续内存优化策略设计提供数据基础。

2. 目标 2：使用虚拟内存相关技术减少运行时物理内存量

基于访存行为分析结果，利用虚拟内存相关技术，将模型权重与 KV Cache 转变为应用层可控的流式驻留对象。通过按需加载避免将全部模型参数载入物理内存，并通过智能驱逐策略及时释放短期内不会被访问的数据页，从而降低运行时物理内存峰值。

3. 目标 3：通过预取等技术掩盖 I/O 延迟，提升推理性能

利用 LLM 推理“逐层顺序访问”的可预测性，设计并实现异步预取调度器。通过独立的后台加载线程池，在推理线程执行当前层计算的同时提前加载未来层参数，实现计算与 I/O 的重叠，有效隐藏存储访问延迟。

1.3 项目实现内容

针对上述目标，项目制定的实现内容如下：

1. 开发环境搭建

在 Ubuntu 22.04/24.04 发行版上搭建 `llama.cpp` 开发环境，配置必要的编译工具链和性能测试工具。使用 `cgroup` 和 `taskset` 模拟内存受限环境，确保开发环境能够复现资源受限场景。

2. LLM 推理访存特征分析

基于 `llama.cpp` 源码和 GGUF 模型文件格式，分析模型参数的文件布局和内存布局，统计各层张量大小分布。通过日志埋点和 `perf` 工具采集推理过程中的访存轨迹，验证逐层顺序访问行为，并测量 KV Cache 随上下文长度增长的内存变化规律。

3. 权重流式加载模块设计与实现

从运行时模型权重加载的行为分析出发，实现 MoE 模型以专家为粒度、Dense 模型以层为粒度的权重按需加载与驻留控制机制，并实现内存压力约束下动态调整权重常驻范围。

4. 异步预取调度模块设计与实现

构建后台加载机制，将权重读取从主推理路径中解耦，实现预取与计算的并行执行，以降低同步 I/O 对推理过程的影响。

5. 内存驱逐与缓存管理模块设计与实现

基于访问局部性与使用频率设计权重与中间状态的驱逐策略，通过分级保留与按需替换机制，在内存受限条件下维持系统稳定运行。

6. KV Cache 统一管理模块设计与实现

对 KV Cache 引入运行时回收策略，对非活跃上下文进行按需释放与恢复管理，从而降低多会话与长上下文场景下的内存占用。

7. 自适应调优模块设计与实现

根据运行时内存压力与系统负载动态调整权重驻留规模与预取强度，在性能开销与内存占用之间实现自适应平衡。

8. 系统集成测试与验证

将上述模块集成到 `llama.cpp` 推理流程中，端到端打通从模型加载到文本生成的全流程，并在多个模型上进行正确性、内存占用和性能基准测试。

1.4 项目完成情况

在初赛阶段，项目已完成 LLM 推理内存优化系统核心功能的设计与实现。系统实现了滑动窗口内存管理、异步预取调度、智能驱逐、权重流式替换、KV Cache 运行时物理页回收等核心模块，并成功将各模块集成至 `llama.cpp` 推理框架中运行。同时，项目对系统的内存节省效果和推理性能进行了初步测试。

表 2: 实现内容完成情况

实现内容	对应目标	完成情况	说明
开发环境搭建	目标 1、2、3	完成	完成 <code>llama.cpp</code> 编译部署，配置 GCC/CMake 与 <code>perf</code> 工具，并通过 <code>cgroup v2</code> 与 <code>taskset</code> 构建内存与算力受限测试环境。

实现内容	对应目标	完成情况	说明
LLM 推理访存特征分析	目标 1	完成	基于 GGUF 文件结构与 llama.cpp 运行过程分析参数布局与访存轨迹，验证逐层顺序访问特征及 KV Cache 随上下文增长的线性扩展规律。
权重流式加载模块设计与实现	目标 2	完成	实现 Dense 模型层级与 MoE 模型专家级权重流式加载机制，在内存压力下动态调整驻留权重集合。
异步预取调度模块设计与实现	目标 3	完成	利用 CLG 预测机制驱动后台线程池调度，并通过 O_DIRECT + pread 实现异步加载，使预取与计算并行以隐藏 I/O 延迟
内存驱逐与缓存管理模块设计与实现	目标 2	完成	基于访问距离与访问频率构建 LRU + 热度保护混合策略，实现权重与中间状态的分级驱逐与内存回收控制。
KV Cache 统一管理模块设计与实现	目标 1、2	完成	引入 block-level KV 管理与 swap 机制，结合 madvise(MADV_DONTNEED) 实现非活跃 KV 页释放与 resume 恢复。
自适应调优模块设计与实现	目标 3	完成	基于 cgroup memory.current 与系统负载动态调整驻留预算与预取深度，实现内存压力感知的自适应调度策略。
系统集成测试与验证	目标 1、2、3	完成	完成端到端集成测试，在多模型与不同上下文下验证 RSS 控制、KV 回收正确性及推理吞吐稳定性。

1.5 项目开发计划

表 3: 项目开发计划

预期内容	预期时间
项目初步调研：可行性分析、技术选型、llama.cpp 源码调研与运行机制分析	第 1 周
项目深度调研：FlexInfer 论文复现、llama.cpp 权重加载与 KV Cache 管理机制深入分析	第 2 周
开发环境搭建、系统架构设计、模块接口定义、初赛目标设定	第 3 周
原型验证：实现基础权重流式加载机制（MoE/Layer）、KV Cache 基础管理与同步预取机制，完成功能正确性验证与初步内存优化效果测量	第 4-5 周
异步优化：实现基于 CLG 的异步预取调度器与后台加载线程池，完成计算与 I/O 并行优化，并进行性能基准测试与参数调优	第 6-7 周
生产就绪：完善异常处理与恢复机制，引入自适应调优策略，构建完整测试体系与文档示例	第 8 周
性能测试、内存占用测试、与 mmap 基线及相关优化方法进行对比分析	第 9 周
初赛文档撰写、代码仓库整理、初赛资料与答辩材料准备	第 10 周
与赛题导师沟通决赛方向，制定基于系统优化的决赛阶段实现方案	决赛阶段初期
结合硬件访存特性（带宽瓶颈、NUMA 架构与 I/O 路径）优化权重与 KV Cache 数据流动效率，提升推理吞吐性能	决赛阶段
优化权重与 KV Cache 协同使用机制，提升多模型与长上下文场景下的资源利用效率	决赛阶段
引入异步回收与冷热感知机制，实现 KV Cache 分级管理与动态调度优化	决赛阶段
系统扩展与迁移：将优化框架扩展至更大规模模型及多种 LLM 架构推理场景	决赛阶段

1.6 项目分工

表 4: 项目分工

小组成员	分工内容
苏安炫	负责 LLM 推理权重侧优化相关工作，包括 llama.cpp 框架与 GGUF 模型格式调研，FlexInfer 及权重 offloading 技术分析，权重流式加载模块与异步预取调度模块的设计与实现，并负责权重相关模块的性能测试与优化。
李思甜	负责 KV Cache 与运行时内存管理相关工作，包括 LLM 推理访存行为与 mmap 机制调研，KV Cache 管理机制与缓存驱逐策略分析，智能驱逐模块与 KV Cache 统一管理模块的设计与实现，并负责长文本压力测试与系统验证。

2 现有研究调研概况

本节主要介绍和项目有关的大语言模型推理技术、内存管理方案以及边缘部署优化方案的研究情况，并根据调研结果寻找本项目的切入点。

2.1 LLM 推理的访存行为特征

LLM 推理过程具有独特的访存行为特征，这是本项目所有优化设计的出发点。LLM 推理通常分为 Prefill 阶段和 Decode 阶段。

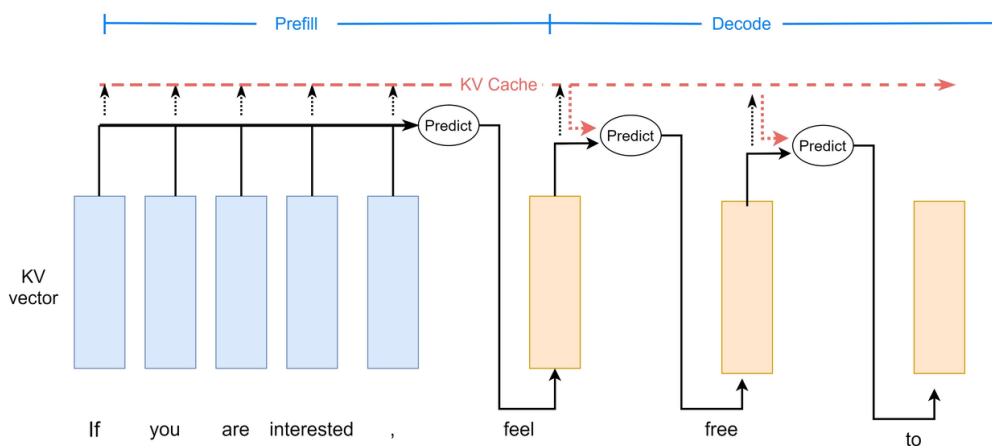


图 1: LLM 推理中的 Prefill 与 Decode 阶段

Prefill 阶段并行处理全部输入 Token，在每个注意力层为每个 Token 计算 Key 和 Value 向量，通常具有较高并行度。Decode 阶段以自回归方式逐 Token 生成，每个新

Token 需要对所有先前 Token 的 Key-Value 对进行注意力计算，因而更容易受内存带宽限制。

在访存行为方面，LLM 推理过程中的数据搬运可分为三个独立的 I/O 流：

1. 权重 I/O

每解码一步需要加载模型权重。对于大规模模型而言，权重 I/O 是推理过程中最主要的数据搬运来源之一。权重 I/O 与批次大小无关，可通过批处理进行摊销。

2. KV Cache I/O

每层每序列需要读取历史 Token 的键值缓存。KV Cache I/O 随序列长度和批次大小线性增长，在长上下文场景下可能接近甚至超过模型权重本身。

3. 激活 I/O

激活 I/O 主要来自中间激活张量在 kernel 启动间的读写。Decode 阶段的激活 I/O 通常远小于权重 I/O 和 KV Cache I/O；Prefill 阶段若不使用 FlashAttention，则可能产生较高的中间读写开销。

上述分析表明，权重 I/O 和 KV Cache I/O 是 Decode 阶段的两大主要瓶颈。两者之间存在动态平衡关系：权重 I/O 可通过批次处理摊销，但 KV Cache I/O 随批次和上下文长度线性增长。这一关系是后续优化方案所面临的核心挑战，也是本项目统一管理权重与 KV Cache 的动机所在。

2.2 推理框架

2.2.1 llama.cpp 推理框架

llama.cpp 是一个纯 C/C++ 实现的大语言模型推理引擎，旨在为广泛的硬件设备提供推理能力，从智能手机等边缘设备到多 GPU 服务器集群均可覆盖。其具有如下特点：

1. 纯 C/C++ 实现，零依赖，无需引入 PyTorch 等深度学习框架。
2. 量化作为一等公民，原生内置多种量化格式。
3. 支持 CPU 与 GPU 混合推理，当模型大于显存时可自动拆分至 CPU 内存和 GPU 显存。

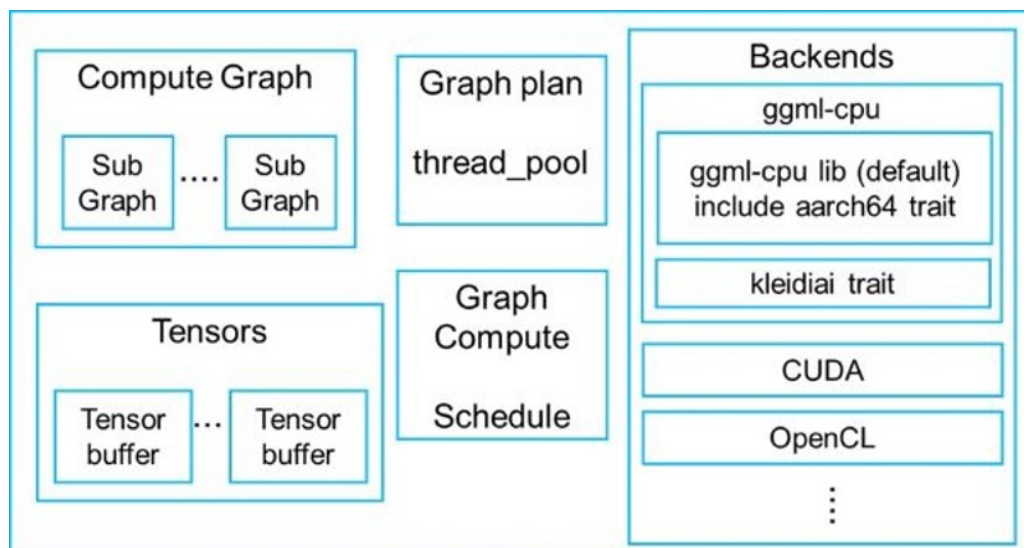


图 2: llama.cpp 推理框架架构

llama.cpp 采用分层架构设计：GGML 层提供底层张量计算、内存管理和后端抽象；llama 核心层实现模型加载、分词、KV Cache 管理、计算图构建及采样功能；工具层提供 CLI、HTTP server 和基准测试工具等。

llama.cpp 当前通过 mmap 机制实现模型参数按需加载。当推理线程访问某层参数时，操作系统通过缺页中断从存储设备加载相应数据页。该机制实现简单，但在物理内存不足时，几乎每次权重访问都会触发 I/O，导致推理吞吐显著下降。本项目使用 llama.cpp 作为基础框架进行二次开发，在其数据加载路径中插入滑动窗口、异步预取、权重流式替换和 KV Cache 回收逻辑。

2.2.2 vLLM 推理框架

vLLM 是一个面向数据中心的高吞吐量 LLM 推理框架，其主要贡献是提出了 PagedAttention 机制。

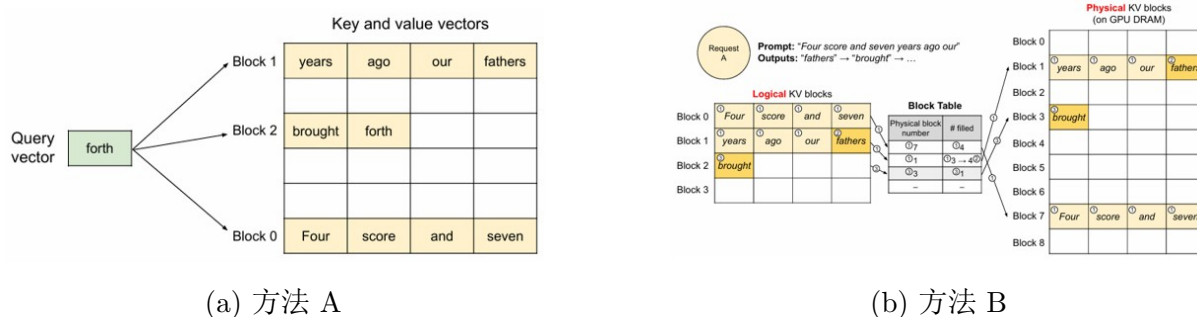


图 3: PagedAttention 机制示意图

PagedAttention 将 KV Cache 切分为固定大小的页或 block，按需分配，其思想与操作系统虚拟内存页表相似。一个 block table 将逻辑页映射到物理内存，使 KV Cache

可非连续存储。PagedAttention 还支持前缀缓存，即多个请求共享同一前缀时，对应的 KV Cache 页可直接复用而非重新计算。vLLM 主要面向 GPU 服务器环境，但其将 KV Cache 视为可灵活管理内存资源的思想，为本项目在边缘端设计 KV Cache 管理策略提供了参考。

2.3 内存受限场景下的优化方案

由前文访存行为分析可知，内存压力主要来源于两类对象：模型权重与运行时KV缓存。前者对应参数存储与传输成本，后者对应生成过程中持续增长的内存消耗。因此，内存优化可以从权重 I/O 与 KV cache I/O 两个方向进行探索。

2.3.1 权重 I/O 优化：模型参数的存储与访问

权重开销主要由模型规模决定，其核心问题在于参数无法完全驻留在计算设备中，必须在不同存储层之间迁移。当前主流的方向有：

量化压缩 量化通过降低参数数值精度来减少存储体积，从而降低加载与传输开销。常见做法是将浮点权重映射到低比特整数表示，并在推理阶段进行近似计算。该类方法的特点是压缩比例在部署阶段基本固定，一旦确定量化策略，模型的内存占用也随之固定，因此难以根据实时资源情况进行调整。

分层存储与按需加载 另一类方法通过构建多级存储结构，将权重分布在内存与外部存储介质上，并在计算时按需调入。典型实现包括基于操作系统的惰性加载机制，以及基于用户态控制的主动预取策略。前者依赖访问触发数据加载，行为不可控；后者通过预测未来访问路径提前调度权重，从而减少计算等待时间。

2.3.2 KV Cache I/O 优化：运行时状态管理

KV cache 是自回归推理过程中逐步增长的中间状态，其规模随生成长度线性增加，是动态内存压力的主要来源。现有的优化方案有：

分页式管理 通过将缓存划分为固定大小的块，并使用映射结构管理逻辑与物理地址关系，可以避免连续内存分配带来的碎片问题，并支持多个请求的并发执行。这种方式将缓存管理从连续内存模型转变为块级资源调度问题，使得系统能够在有限内存条件下支持更大规模的并发负载。

缓存回收与迁移 在内存不足或负载变化时，可以将部分缓存数据从高速存储迁移至低速存储，或直接释放不再需要的部分内容。驱逐策略通常基于优先级或重要性估计，例如根据访问频率、位置特征或注意力强度进行判断。

缓存压缩 另一类方法通过降低缓存中数值表示的精度来减少存储占用，但由于缓存状态依赖生成历史，其误差会在后续计算中逐步传播，因此需要在压缩率与质量之间进行权衡。

结构性压缩 从模型结构层面减少缓存规模的方法包括减少注意力头数量、共享键值表示，或使用低维隐变量替代显式缓存表示。这类方法可以从源头降低缓存生成量，而不是在生成后进行压缩处理。

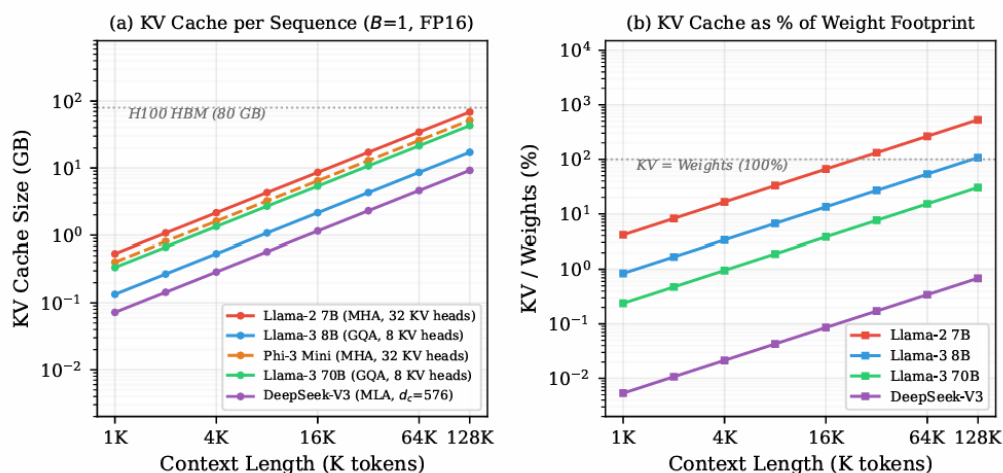


图 4: 架构级 KV 缩减机制

2.4 边缘端部署的特殊挑战

边缘端设备部署 LLM 面临与主流部署环境（如云端、数据中心、服务器集群等）不同的资源约束，其主要挑战体现在以下几个方面：

1. 内存容量严格受限

边缘设备 DRAM 容量通常较小，难以容纳大规模模型权重，即使采用 4-bit 量化仍可能无法完整加载。

2. 存储带宽有限

边缘设备多采用 eMMC 或 UFS 闪存，其顺序读写带宽显著低于主流部署环境中的高性能存储系统，导致模型权重加载与 KV Cache 访问的 I/O 延迟更难被隐藏。

3. 计算能力受限或异构性不足

部分边缘设备仅依赖 CPU 进行推理，或配备的 GPU 计算能力与显存规模有限，难以支撑大规模模型的高效执行。

4. 功耗与散热约束

边缘设备受电池容量与散热条件限制，无法持续维持高功耗计算状态。

上述挑战表明，面向主流部署环境的优化方法难以直接迁移至边缘端，需要针对其存储层次结构与计算资源约束设计专门的系统级优化方案。

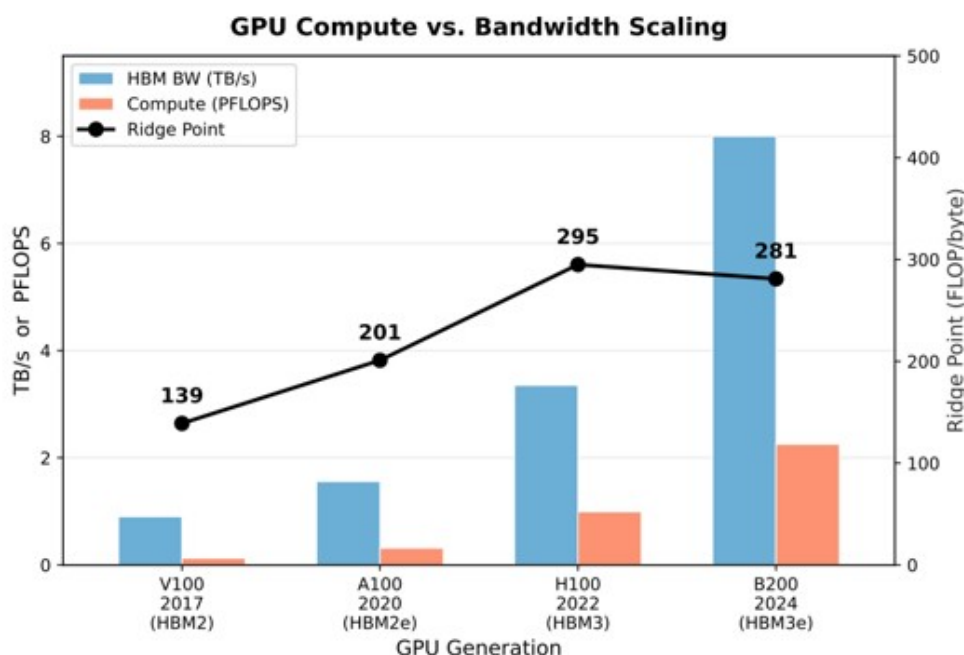


图 5: 边缘端大模型推理资源限制

2.5 现有方案的不足与本项目切入点

综合调研可知，现有方案在内存受限环境下的 LLM 推理中存在以下不足：

1. 权重与 KV Cache 独立管理，未形成统一视图

llama.cpp 通过 mmap 管理权重，通过连续分配管理 KV Cache，两者互不协调。当 KV Cache 增长时，可能挤占权重缓存空间。权重与 KV Cache 在时间维度上存在明确对应关系，但这一关系尚未被充分利用。

2. 权重管理缺乏访存模式感知

现有方案的加载与驱逐策略未充分利用 LLM 推理逐层顺序访问的确定性和可预测性。mmap 方案是被动加载，缺少主动预测；已有预取方案虽具主动性，但未建立显式的层级访问预测模型。

3. KV Cache 管理未适配边缘端 I/O 层次

现有 KV Cache 管理方案多面向 GPU 环境设计，未充分考虑边缘端 DRAM 到 SSD 的 I/O 特性。当 KV Cache 换出至 SSD 时，如何预取、何时换入、如何与权重加载共享 I/O 带宽，仍缺乏系统级方案。

上述问题的核心在于，现有方法通常将权重 I/O 与 KV Cache I/O 作为两个独立优化目标分别处理，而未考虑二者在推理过程中对内存容量与 I/O 带宽的共同竞争关系。这导致单点优化难以从系统层面消除瓶颈。

Configuration	W (GB)	K (GB)	Total (GB)	Dom. Flow	Est. TPOT (ms)
FP16 baseline	141	42	183	W	68
+ INT4 weights (AWQ)	35	42	77	K	29
+ INT4 KV cache (KIVI)	35	10.5	45.5	W	17
+ 2:4 sparsity	17.5	10.5	28	W	10.4
+ Spec. decoding ($\gamma=5, \alpha=0.8$)	4.4*	10.5	14.9	K	5.6

图 6: 权重 I/O 与 KV Cache I/O 孤立优化的瓶颈效应

基于上述分析，本项目构建统一的运行时内存管理框架，将模型权重与 KV Cache 纳入同一调度体系。在该框架下，以层、MoE 专家以及 KV block 为基本管理粒度，对权重驻留状态与 KV Cache 状态进行联合管理；结合 CLG 预测机制实现跨层预取调度，并通过异步加载与驱逐策略实现 I/O 与计算的重叠执行。当权重或 KV block 进入非活跃状态时，系统将其纳入可回收集合，并在资源压力或预测触发下执行换入与换出操作，从而在内存受限环境下实现更稳定的推理执行与更高的资源利用效率。

3 系统总体设计

本节从系统整体架构出发，对本项目的设计理念、系统结构、核心模块以及运行时交互机制进行说明。系统围绕 LLM 推理过程中的权重 I/O 与 KV Cache I/O 问题，构建统一的运行时内存管理框架，在边缘端资源受限条件下实现计算与数据访问的协同优化。

3.1 设计理念

针对 LLM 推理在内存与 I/O 双重受限环境下的执行特点，本系统设计理念如下：

1. 可预测的访存调度

利用 LLM 逐层顺序执行与 MoE 稀疏激活特性，构建面向可预测访存模式的调度策略，而非通用调度策略。

2. 隐藏数据加载延迟

通过异步预取与后台 I/O 执行机制，将数据加载从推理主线程中解耦，实现计算与存储访问的并行执行。

3. 降低峰值物理内存占用

通过流式加载与动态换出机制，对模型权重与 KV Cache 进行统一管理，在不影响正确性的前提下显著降低峰值物理内存占用。

4. 动态适配运行时资源

支持根据内存预算与 I/O 带宽动态调整权重驻留规模、预取强度与 KV Cache 回收策略。

5. 保证推理语义不变

所有内存优化操作均不改变框架底层算子，保证输出结果正确性。

3.2 系统架构

系统整体由三层组成：推理执行层、运行时内存调度层以及底层虚拟内存与存储层。其中，运行时内存调度层是核心控制模块，负责协调权重 I/O 与 KV Cache I/O 的统一管理。

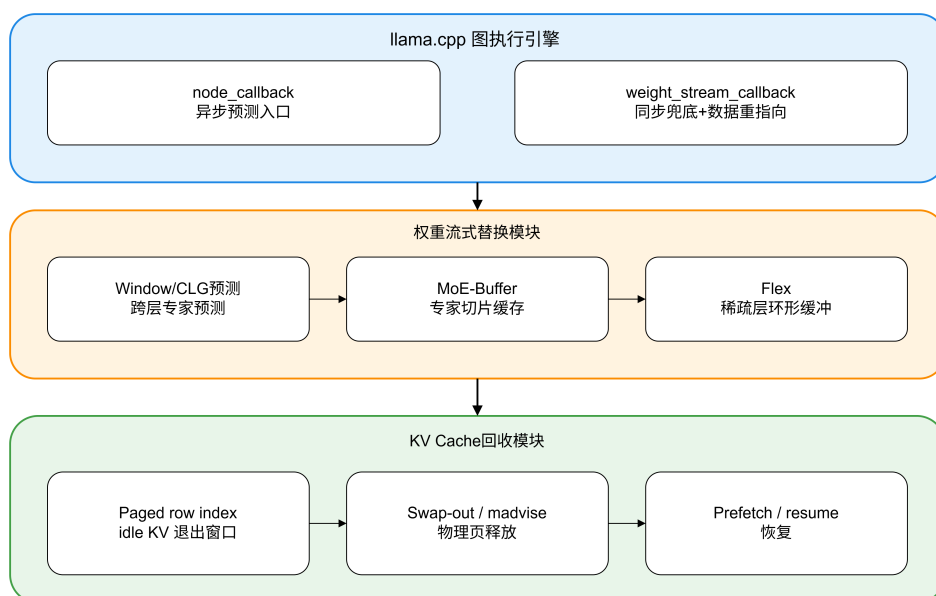


图 7: 系统总体架构图

1. 推理执行层

该层负责 LLM 前向计算过程的执行，包括 Transformer 层计算、MoE 路由选择以及 attention 计算等核心算子。执行过程中通过 `node_callback` 与调度层进行交互，获取权重与 KV Cache 的驻留状态，从而决定是否触发加载或等待。

2. 运行时内存调度层

该层为系统的核心控制单元，负责权重与 KV Cache 的统一管理与调度。其主要功能包括：

- 基于 Window/CLG 的跨层预测，实现 MoE 专家与后续层权重的预取调度；
- 管理 MoE Buffer 与 Dense Flex 缓冲区，实现权重流式驻留与替换；
- 维护 KV Cache block 状态机，协调 idle block 的回收与恢复；
- 通过异步 worker 调度 I/O 操作，实现计算与数据访问重叠。

该层通过统一的状态管理机制，将权重 I/O 与 KV Cache I/O 纳入同一调度框架，避免独立优化带来的资源竞争问题。

3. 虚拟内存与存储层。

该层由操作系统虚拟内存管理与外部存储介质构成，提供底层数据承载能力，包括 `mmap` 映射机制、页缓存管理以及 SSD / DRAM 之间的数据迁移能力。系统通过 `madvise` 与 `O_DIRECT` 等机制与该层交互，实现对物理页分配与释放的显式控制。

在整体运行过程中，推理执行层发起计算请求并触发调度信号，运行时内存调度层根据预测结果与状态信息决定权重与 KV Cache 的加载与回收策略，并通过异步 I/O 操作与底层存储层进行数据交互。三层之间形成“计算驱动调度、调度驱动 I/O”的闭环执行结构，从而在内存受限条件下实现计算与数据访问的协同优化。

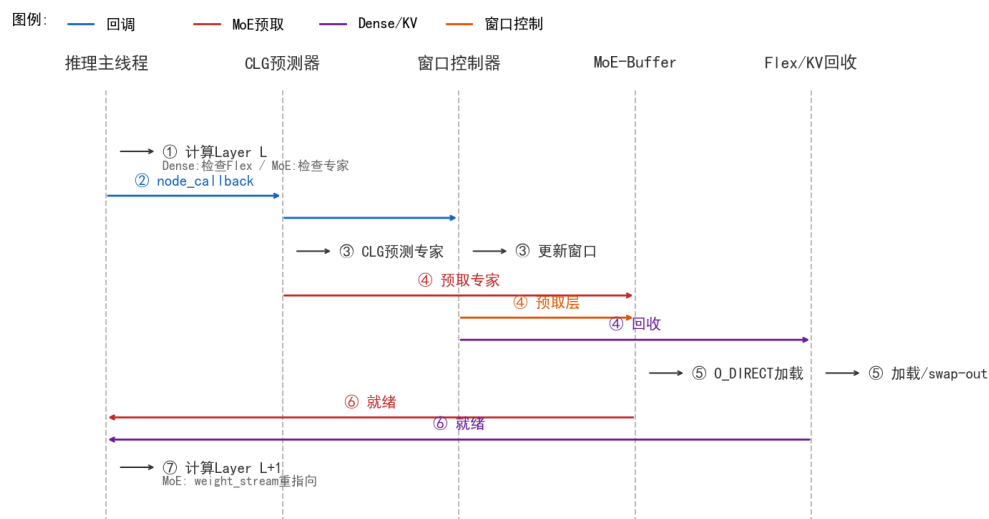


图 8: 模块交互时序图

3.3 系统核心组成

3.3.1 权重管理模块

权重管理模块统一负责模型参数的流式加载、驻留控制与动态替换，其核心目标是在内存受限环境下，以最小化常驻内存占用为目标，通过预测驱动实现权重 I/O 与计算的重叠执行。在实现机制上，权重管理包含两条执行路径：

- **Dense 权重路径：**以 layer 为基本单位，通过环形缓冲区（Flex Buffer）维护驻留集合。在窗口范围内的 layer 保持常驻状态，窗口外 layer 通过 LRU 策略进行替换与回收；当发生缺页时，由预取线程进行异步加载以避免阻塞计算线程。
- **MoE 权重路径：**以 expert 为基本单位，通过 CLG 预测得到下一层可能激活的专家集合，并由后台 worker 通过 `0_DIRECT` 异步加载至受控缓冲区（MoE Buffer）。系统通过状态机维护 expert 的 COLD / INFLIGHT / RESIDENT 状态，并在推理过程中通过指针重定向机制使 GEMM 操作直接访问缓冲区权重，从而避免对 mmap 区域的直接访问。

在整个执行过程中，权重加载、预取与驱逐均由后台 worker 线程池执行，实现计算与 I/O 的解耦与重叠。

3.3.2 KV Cache 管理模块

KV Cache 管理模块负责对自回归生成过程中持续增长的 attention 状态进行运行时管理。与模型权重不同，KV Cache 是推理过程中动态产生的状态数据，其生命周期与 sequence、position、session 活跃状态以及 attention read window 密切相关。因此，系统并不简单地释放历史 token，而是在保证当前 active request 可见 KV 完整性的前提下，只对 idle-owned 且 active-invisible 的 KV block 执行回收。

系统以 KV block 为基本管理粒度，将连续 KV buffer 划分为固定大小 block，并为每个 block 维护如下状态信息：

- **Owner 信息：**记录该 block 所属 sequence / session，用于区分 active-owned、idle-owned 和共享 block；
- **驻留状态：**区分 RESIDENT、SWAPPED 等状态，表示该 block 的物理页是否仍驻留在 KV buffer 中；
- **访问可见性：**判断该 block 是否仍处于当前 active attention read window；
- **恢复保护状态：**对 resume-pending 或刚完成 prefetch 的 block 设置保护，避免恢复后立即被再次换出。

对于满足安全条件的 idle block，系统先将其 K/V 数据写入 backing store，记录恢复所需的 offset 与 size，然后对原 KV tensor 所覆盖的页执行 `madvise(MADV_DONTNEED)`。当对应 session 后续恢复时，系统再通过 swap-in 或分步 prefetch 将 block 恢复到原 KV buffer 中。该设计使 KV Cache 从“逻辑上仍存在”与“物理页是否驻留”两个维度解耦，从而在长上下文和多会话场景下降低当前 RSS。

3.3.3 统一调度与执行机制

系统在整体架构上将权重 I/O 与 KV Cache I/O 作为两类共同竞争内存容量和存储带宽的运行对象进行分析，但二者在当前实现中采用相对独立的执行路径。权重侧主要依赖 `node_callback`、`weight_stream_callback` 和后台 I/O 线程完成预测、预取和同步兜底；KV Cache 侧则主要依赖 trace replay / driver 层提供的 session idle、resume-pending 和 active decode 时机，在这些调度点上触发 idle maintenance、swap-out、resume-aware prefetch 和 final sync。

因此，当前系统实现的是统一资源视角下的双路径协同，而不是将权重与 KV 完全合并到同一个 worker pool 中。具体而言：

- 权重模块负责控制模型参数页或匿名缓冲区的驻留集合，降低权重侧 RSS 与 page cache / repack 冗余；
- KV Cache 模块负责控制 idle session 历史 KV block 的物理驻留状态，降低长上下文、多会话场景下的运行时 KV RSS；
- 二者通过统一的测试框架、环境变量开关和 trace replay workload 进行消融与组合验证，从而评估 weight-only、KV-only 以及 combined 模式下的内存收益与性能开销。

该设计避免了单点优化只转移瓶颈的问题：权重侧优化降低模型参数驻留压力，KV 侧优化降低运行时上下文状态驻留压力，二者分别作用于不同内存来源，因此在 Dense 模型综合测试中能够表现出可叠加的 RSS 下降效果。

4 模块设计与实现

本节具体介绍本项目的设计和实现，主要内容包括双钩子接口、层索引构建、滑动窗口机制、预取与回收机制、权重流式替换模块、KV Cache 运行时回收模块、正确性保证、配置参数以及与 `llama.cpp` 原有功能的兼容性。

4.1 双钩子接口

系统在 GGML CPU 后端中新增两个最小侵入钩子，作为所有流式机制的基础设施：`node_callback` 和 `weight_stream_callback`。前者在图节点完成后触发，适合执

行异步预测和预取调度；后者在算子 kernel 执行前通过 barrier 同步触发，适合执行权重指向和同步正确性兜底。

二者分别构成“异步调度路径”和“同步执行路径”，在系统层面形成预测驱动与执行一致性的双层控制结构。

4.1.1 node_callback

node_callback 在图节点完成后触发，不阻塞计算线程，适合执行预测和预取调度等后台任务。该钩子以函数指针形式注册到 GGML CPU 后端，每个计算节点执行完毕后、线程 barrier 之后，由 `ith == 0` 的线程调用。

```

1 // ggml/include/ggml-cpu.h
2 typedef void (*ggml_graph_compute_sequence_node_callback)(
3     const ggml_tensor * node,
4     void * user_data
5 );
6 // ggml/src/ggml-cpu/ggml-cpu.c
7 static void ggml_cpu_graph_compute(...) {
8     for (int node_n = 0; node_n < cgraph->n_nodes; node_n++) {
9         // compute node ...
10        // barrier ensures all threads finish current node
11        if (node_n + 1 < cgraph->n_nodes || tp->sequence_node_callback !=
12            NULL)
13            ggml_barrier(state->threadpool);
14
15        // only thread 0 triggers callback
16        if (state->ith == 0 && tp->sequence_node_callback != NULL) {
17            tp->sequence_node_callback(node, tp->sequence_callback_data);
18        }
19    }
20 }
```

在 llama.cpp 上层，系统注册 llama_window_node_done 作为回调函数。该函数从节点名中解析层号，例如从 `ffn_inp-5` 或 `blk_5_attn` 中解析出层号 5，然后调用窗口推进逻辑或 CLG 预测调度逻辑。

```

1 // src/llama-window.cpp
2 void llama_window_node_done(llvm_window_context& ctx,
3     const ggml_tensor* node) {
```

```

4     const char* name = ggml_get_name(node);
5     int layer = parse_layer_from_node_name(name);
6
7     if (layer < 0 || layer >= ctx.n_layers) return;
8
9     int prev = ctx.observed_layer.exchange(layer,
10    std::memory_order_relaxed);
11    if (prev == layer) return;
12
13    std::lock_guard<std::mutex> lock(ctx.mutex);
14
15    if (ctx.graph_active) {
16        llama_window_advance_locked(&ctx, layer, true);
17
18        if (ctx.clg_enabled) {
19            llama_window_clg_predict_and_schedule(&ctx, layer);
20        }
21    }
22 }
23

```

该回调的设计重点在于“轻量”和“不阻塞”。它只负责捕获图执行进度、解析当前层号并派发调度请求，不直接执行重型 I/O 操作，因此不会显著影响推理主线程吞吐。

4.1.2 weight_stream_callback

`weight_stream_callback` 在算子 kernel 执行前通过 barrier 同步触发，用于权重数据的重指向和同步正确性兜底。该钩子主要服务于 MoE 模型中的 `mul_mat_id` 算子。所有线程到达 barrier 后，由 `ith == 0` 的线程执行实际权重重指向操作，随后再次进入 barrier，确保所有线程在权重就绪后再进入计算 kernel。

```

1 // ggml/include/ggml-cpu.h
2 typedef bool (*ggml_cpu_weight_stream_callback)(
3 ggml_tensor * op,
4 int ith,
5 void * user_data
6 );
7

```

```

8 // ggml/src/ggml-cpu/ggml-cpu.c
9 static void ggml_cpu_graph_compute(...) {
10     for (int node_n = 0; node_n < cgraph->n_nodes; node_n++) {
11         ggml_tensor * node = cgraph->nodes[node_n];
12
13         if (tp->weight_stream_callback != NULL) {
14             ggml_barrier(state->threadpool);
15
16             if (state->ith == 0) {
17                 tp->weight_stream_callback(
18                     node,
19                     state->ith,
20                     tp->weight_stream_user_data
21                 );
22             }
23
24             ggml_barrier(state->threadpool);
25         }
26
27         // execute kernel ...
28     }
29 }
30

```

在 llama.cpp 上层，系统注册 llama_moe_buffer_stream_callback 作为 MoE 专家权重流式回调函数。该函数只处理 GGML_OP_MUL_MAT_ID 算子，检查当前算子实际选中的专家是否已经驻留在 MoE-Buffer 中；若未驻留，则触发同步加载或等待已有加载完成；随后将专家权重张量的 data 指针重定向至匿名缓冲区。

```

1 // src/llama-moe-buffer.cpp
2 bool llama_moe_buffer_stream_callback(
3     ggml_tensor * op,
4     int ith,
5     void * user_data
6 ) {
7     if (op->op != GGML_OP_MUL_MAT_ID) return false;
8
9     auto* ctx = static_cast<moe_buffer_context*>(user_data);

```



```

10     ggml_tensor * exps = op->src[0];
11
12     auto it = ctx->by_name.find(ggml_get_name(exps));
13     if (it == ctx->by_name.end()) return false;
14
15     moe_managed & m = it->second;
16
17     if (ith == 0) {
18         moe_ensure_buf(m);
19
20         // redirect pointer to managed buffer
21         exps->data = m.buf;
22
23         ggml_tensor * ids = op->src[2];
24         const int32_t * idp = (const int32_t *) ids->data;
25
26         for (int64_t i = 0; i < ggml_nelements(ids); ++i) {
27             moe_stream_slice(*ctx, m, (int) idp[i]);
28         }
29     }
30
31     return true;
32 }
33

```

表 5: 双钩子接口对比

钩子	触发时机	同步方式	用途
node_callback	节点计算完成后	不阻塞计算线程	异步预测、预取调度、窗口推进
weight_stream_callback	算子执行前	barrier 同步	权重重指向、同步加载兜底、正确性保证

二者共同构成“异步预取 + 同步兜底”的双层保障机制：前者负责预测与调度，后者负责执行一致性与正确性收敛。

4.2 层索引与滑动窗口

系统在模型加载完成后构建层级索引，并在此基础上实现滑动窗口机制，用于驱动后续的预取与回收调度。该部分由三部分组成：层索引构建、滑动窗口定义与推进调度，以及基于 `madvise` 的页级内存操作。

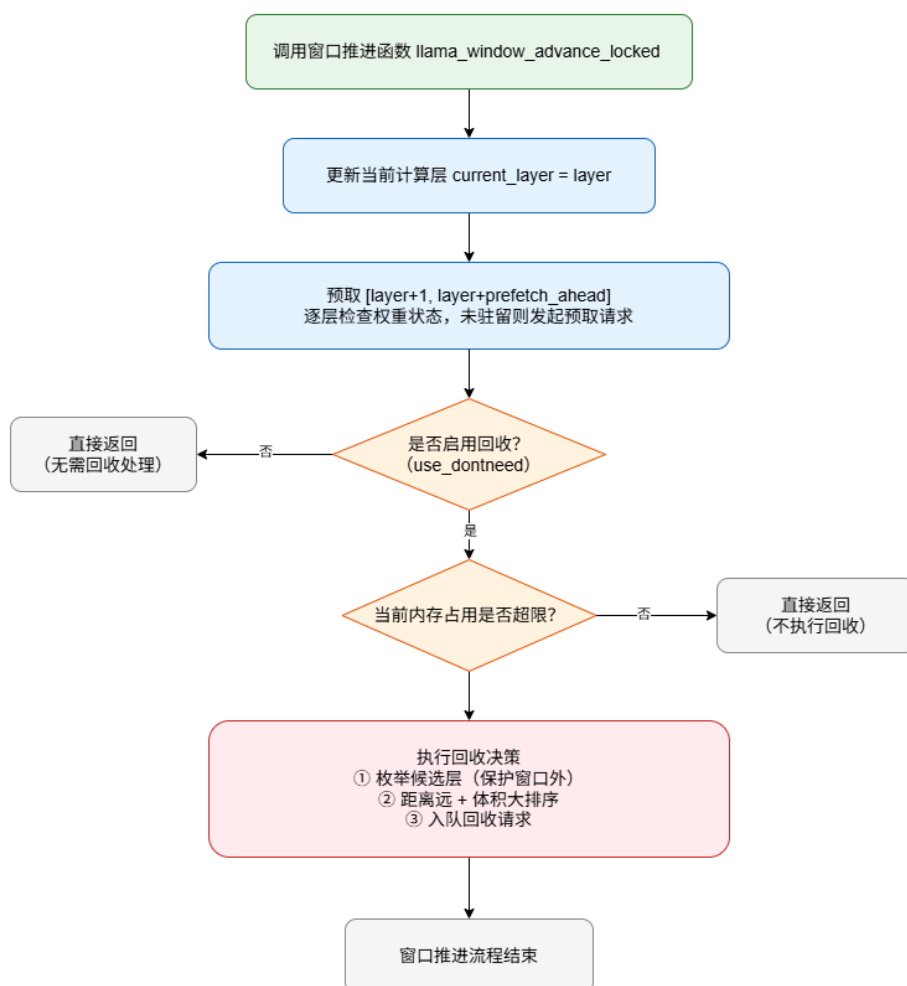


图 9: 窗口推进调动流程图

4.2.1 层索引构建

系统首先遍历 GGUF 模型中的所有权重张量，将其地址区间映射到对应的模型层。该过程的目标是建立 “layer $\rightarrow$ memory range” 的映射关系，为后续页级调度提供基础。

通过解析张量名称获取层号，例如 `blk.5.attn.q.weight` 映射到第 5 层。

```

1 int layer = parse_layer_from_name(tensor_name);
2

```

随后对每个权重张量的地址进行页对齐处理：

```

1 uintptr_t start = (uintptr_t)addr & ~(page_size - 1);
2 uintptr_t end   = ((uintptr_t)addr + size + page_size - 1)
3 & ~(page_size - 1);
4

```

页对齐的原因在于 `madvise`、缺页调入以及物理页回收均以页为基本单位。如果不进行对齐，可能导致跨张量页被误回收或错误驱逐。

4.2.2 跨层共享页检测

在构建区间集合后，系统进一步执行跨层共享页检测。原因在于不同层的权重可能映射到同一物理页，如果直接回收其中一层，会影响另一层的正确性。

系统通过扫描排序后的区间集合，检测是否存在跨层重叠：

```

1 for (size_t i = 0; i < candidates.size(); ++i) {
2     uintptr_t end_i =
3     (uintptr_t)candidates[i].range.addr +
4     candidates[i].range.size;
5
6     for (size_t j = i + 1; j < candidates.size(); ++j) {
7         if ((uintptr_t)candidates[j].range.addr >= end_i) break;
8
9         if (candidates[i].layer != candidates[j].layer) {
10             candidates[i].safe = false;
11             candidates[j].safe = false;
12         }
13     }
14 }
15

```

被标记为 `safe=false` 的区间不会参与后续主动回收。

4.2.3 滑动窗口定义与语义

滑动窗口机制用于限制“当前活跃层附近”的驻留范围。

设模型层数为 N ，当前层为 L ，向后保留 B 层，向前预取 A 层，则窗口定义为：

$$P(L) = \{(L - B) \bmod N, \dots, L, \dots, (L + A) \bmod N\} \quad (1)$$

窗口大小为：

$$W = A + B + 1 \quad (2)$$

窗口内层保持驻留或优先驻留，窗口外层允许被回收。

4.2.4 窗口保护谓词

系统通过谓词函数判断某一层是否属于保护窗口：

```

1 static bool llama_window_is_protected(
2     int layer,
3     int current,
4     int n_layers,
5     int keep_behind,
6     int prefetch_ahead
7 ) {
8     if (layer < 0 || current < 0 || n_layers <= 0) {
9         return true;
10    }
11
12    for (int delta = -keep_behind;
13         delta <= prefetch_ahead;
14         ++delta) {
15
16        int p = (current + delta) % n_layers;
17        if (p < 0) p += n_layers;
18
19        if (layer == p) return true;
20    }
21
22    return false;
23 }
24
```

该函数定义了窗口内层不可回收的硬约束条件。

4.2.5 窗口推进调度

每当完成一层计算后，系统调用窗口推进函数更新状态，并触发预取与回收逻辑。

```

1 void llama_window_advance_locked(

```

```

2 llama_window_context* ctx,
3 int layer,
4 bool from_graph
5 ) {
6     ctx->current_layer = layer;
7
8     // -----
9     // Prefetch future layers
10    // -----
11    for (int delta = 1;
12         delta <= ctx->params.prefetch_ahead
13         && ctx->prefetch_remaining > 0;
14         ++delta) {
15
16        int target = (layer + delta) % ctx->n_layers;
17
18        if (ctx->layers[target].state < LAYER_RESIDENT) {
19            llama_window_request_prefetch_locked(
20                ctx, target, delta
21            );
22        }
23    }
24
25    // -----
26    // Reclaim under memory pressure
27    // -----
28    if (!ctx->params.use_dontneed) return;
29
30    bool pressure =
31        (ctx->params.memory_limit > 0 &&
32         ctx->stats.current_rss >
33         ctx->params.memory_limit * 1024ULL * 1024ULL * 1024ULL);
34
35    if (ctx->params.memory_limit == 0 || pressure) {
36
37        std::vector<int> candidates;
38

```

```

39     for (int c = 0; c < ctx->n_layers; ++c) {
40         if (!llama_window_is_protected(
41             c,
42             layer,
43             ctx->n_layers,
44             ctx->params.keep_behind,
45             ctx->params.prefetch_ahead)) {
46             candidates.push_back(c);
47         }
48     }
49
50     std::sort(candidates.begin(), candidates.end(),
51     [&](int a, int b) {
52         int dist_a = std::abs(a - layer);
53         int dist_b = std::abs(b - layer);
54
55         size_t size_a = ctx->layers[a].total_bytes;
56         size_t size_b = ctx->layers[b].total_bytes;
57
58         return (dist_a * 1024ULL * 1024ULL + size_a / 2)
59             > (dist_b * 1024ULL * 1024ULL + size_b / 2);
60     });
61
62     for (int c : candidates) {
63         llama_window_request_reclaim_locked(ctx, c, 100);
64     }
65 }
66 }
67

```

该调度策略体现两个核心原则：

1. ****时间局部性优先****：距离当前层越远，越可能被回收 2. ****内存收益优先****：体积更大的层优先释放以获得更高内存收益

4.2.6 madvise 预取与回收机制

滑动窗口只决定“策略”，实际页级操作依赖 Linux 虚拟内存接口 `madvise`。

预取路径：

```

1  madvise(addr, size, MADV_POPULATE_READ);
2  madvise(addr, size, MADV_WILLNEED);
3

```

其中优先使用 `MADV_POPULATE_READ`，在旧内核上回退至 `WILLNEED` 并通过逐页 touch 保证驻留。

回收路径：

```

1  madvise(addr, size, MADV_DONTNEED);
2

```

回收操作不会破坏文件映射，仅释放物理页，后续访问可重新从文件页缓存恢复。

4.2.7 窗口 worker 执行模型

预取与回收均由后台 worker 线程执行，避免阻塞推理主线程。

```

1  static void llama_window_worker(llama_window_context* ctx) {
2      while (!ctx->stop_workers) {
3
4          Task task;
5
6          if (!ctx->task_queue.pop(task)) continue;
7
8          if (task.type == TASK_PREFETCH) {
9
10             for (auto &range : ctx->layers[task.layer].ranges) {
11                 llama_window_populate(range.addr, range.size);
12             }
13
14             ctx->layers[task.layer].state = LAYER_RESIDENT;
15         }
16
17         else if (task.type == TASK_RECLAIM) {
18
19             std::lock_guard<std::mutex> lock(ctx->mutex);
20
21             if (!llama_window_is_protected(
22                 task.layer,
23                 ctx->current_layer,

```

```
24         ctx->n_layers,  
25         ctx->params.keep_behind,  
26         ctx->params.prefetch_ahead)) {  
27  
28         for (auto &range : ctx->layers[task.layer].ranges) {  
29             madvise(range.addr, range.size, MADV_DONTNEED);  
30         }  
31  
32         ctx->layers[task.layer].state = LAYER_COLD;  
33     }  
34 }  
35 }  
36 }  
37
```

该设计通过“双重判断机制”保证安全性：

- 入队阶段避免明显无效任务
- 执行阶段再次检查窗口状态防止竞态错误

4.3 Dense 权重环形缓冲

Dense 权重路径采用 Flex Buffer 机制，以 layer 为基本单位构建匿名环形缓冲区，用于替代 mmap 直接驻留策略，从而在内存受限条件下避免 OOM 并降低页抖动。

该机制的核心思想是：在有限 slot 集合上维护“layer $\rightarrow$ slot”映射关系，通过 LRU 替换策略动态调整驻留层集合。

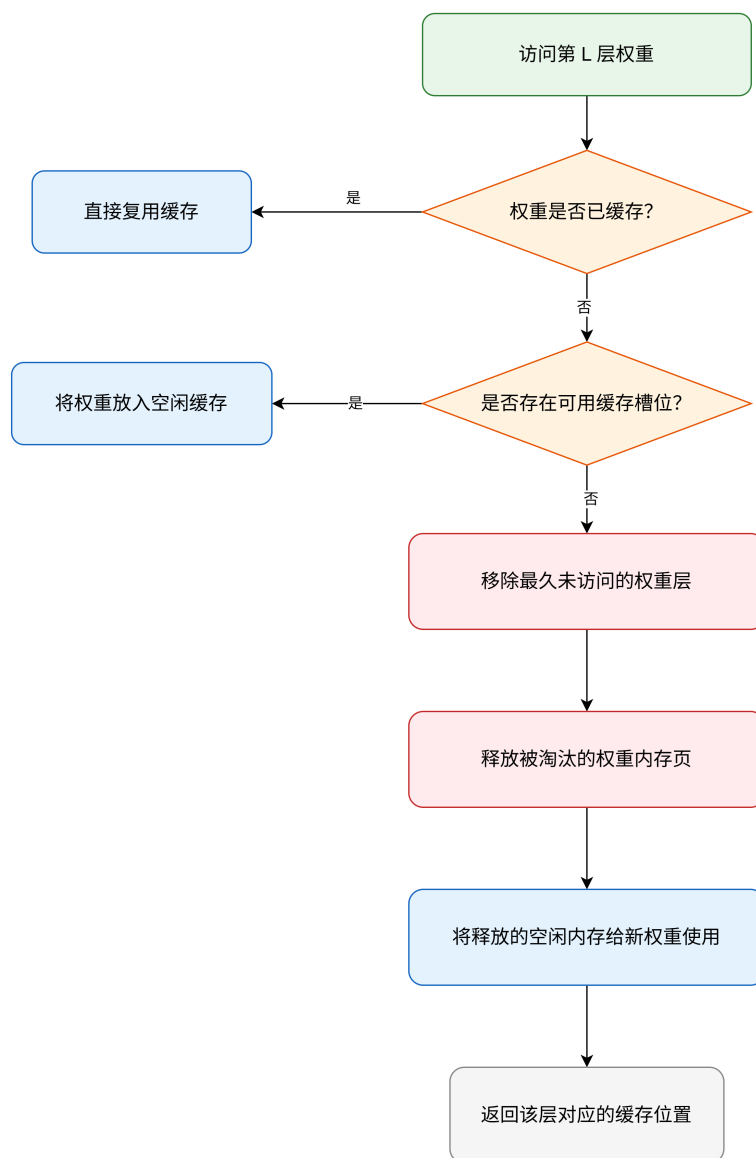


图 10: Dense模型权重管理机制流程图

4.3.1 槽位分配与命中逻辑

系统为 Dense 权重预分配固定数量的匿名槽位，每个槽位可容纳一个 layer 的权重数据。

访问流程如下：

- 若 layer 已绑定 slot，则直接复用

- 若存在空闲 slot，则直接分配
- 否则触发 LRU 驱逐

```

1  if (ctx.layers[layer].slot >= 0) {
2      ctx.layers[layer].last_use = ++ctx.clock;
3      return ctx.layers[layer].slot;
4  }
5

```

当不存在空闲 slot 时，执行替换逻辑：

```

1  for (size_t s = 0; s < ctx.slots.size(); ++s) {
2      if (ctx.slot_layer[s] < 0) {
3          ctx.slot_layer[s] = layer;
4          ctx.layers[layer].slot = s;
5          ctx.layers[layer].state = LAYER_RESIDENT;
6          ctx.layers[layer].last_use = ++ctx.clock;
7          return s;
8      }
9  }
10

```

4.3.2 LRU 驱逐策略

当 slot 已满时，系统基于 LRU 选择最久未访问的 layer 作为驱逐对象：

```

1  int victim = -1;
2  uint64_t oldest = UINT64_MAX;
3
4  for (int l = 0; l < ctx.n_layers; ++l) {
5      if (ctx.layers[l].slot >= 0 &&
6          ctx.layers[l].state == LAYER_RESIDENT &&
7          ctx.layers[l].last_use < oldest) {
8
9          oldest = ctx.layers[l].last_use;
10         victim = l;
11     }
12 }
13

```

驱逐后释放对应匿名映射：

```
1 int slot = ctx.layers[victim].slot;
2
3 ctx.layers[victim].slot = -1;
4 ctx.layers[victim].state = LAYER_NOT_RESIDENT;
5
6 if (ctx.slots[slot].addr) {
7     munmap(ctx.slots[slot].addr, ctx.slots[slot].size);
8     ctx.slots[slot].addr = nullptr;
9 }
10
```

4.3.3 slot 重绑定逻辑

完成驱逐后，将 slot 分配给新 layer，并更新状态：

```
1 ctx.slot_layer[slot] = layer;
2
3 ctx.layers[layer].slot = slot;
4 ctx.layers[layer].state = LAYER_RESIDENT;
5 ctx.layers[layer].last_use = ++ctx.clock;
6
```

该过程确保 slot 在任意时刻只绑定一个 layer，从而避免写冲突与地址污染。

4.3.4 执行语义

Flex Buffer 的执行语义与 mmap 模型不同：

- mmap：依赖内核 page cache，被动调页
- Flex：用户态 slot 管理 + 显式 munmap 驱逐

因此 Flex 在行为上更接近“显式 working set 控制器”，而非传统虚拟内存抽象。

4.3.5 内存释放行为

当 layer 被驱逐时，其对应 slot 会立即释放：

```
1 munmap(ctx.slots[slot].addr, ctx.slots[slot].size);
2 ctx.slots[slot].addr = nullptr;
3
```

该设计确保：

- 物理页立即回收
- 不依赖 page cache 延迟回收
- 在内存压力下具备确定性释放行为

4.3.6 Clock 更新机制

每次访问或分配都会更新全局 clock，用于 LRU 判定：

```
1 ctx.layers[layer].last_use = ++ctx.clock;  
2
```

clock 采用单调递增逻辑，避免时间回退带来的 LRU 不稳定性。

4.3.7 机制总结

Dense Flex Buffer 实现了以下目标：

- 将 mmap 被动页调度替换为 slot 显式控制
- 在内存不足时提供确定性驱逐路径
- 通过 LRU 保持局部性最优近似

该机制作为 Dense 路径的基础内存控制层，与滑动窗口策略共同构成“layer-level working set 管理系统”。

4.4 MoE 专家权重流式加载

MoE 专家权重流式加载模块是整个系统的核心路径之一，其目标是在不改变 MoE 数学计算语义的前提下，将专家权重访问从 mmap 被动调页转变为用户态显式流式管理。

系统由五个关键子机制组成：

- CLG 跨层预测
- MoE-Buffer 三态管理与指针重定向
- O_DIRECT 对齐并行读
- 热专家保护策略
- 自适应预算控制与 repack 关闭

整体形成“预测驱动 + 分片流式加载 + 并行 I/O + 动态内存控制”的闭环。

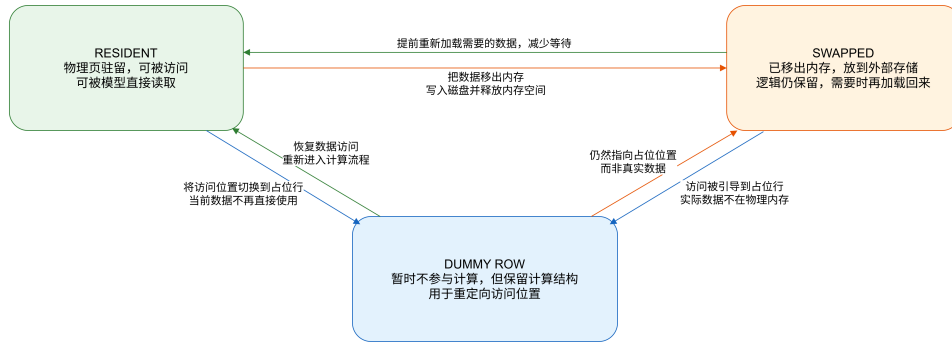


图 11: MoE模型权重管理机制流程图

4.4.1 CLG 预测与专家选择

CLG 模块基于当前隐藏状态预测下一层可能激活的专家集合。

首先对隐藏状态进行 RMSNorm:

$$\hat{h} = \frac{h}{\sqrt{\frac{1}{d} \sum_{i=1}^d h_i^2 + \epsilon}} \odot \gamma \quad (3)$$

随后计算 expert score:

$$s_e = \hat{h}^T W_e \quad (4)$$

最终选择 Top- $K + \delta$ 专家作为预取集合:

$$M = \text{Top}_{K+\delta}(s) \quad (5)$$

实现代码如下:

```

1 // src/llama-window.cpp
2 void llama_window_clg_predict_and_schedule(
3 llama_window_context* ctx,
4 int current_layer
5 ) {
6     float* hs = ctx->hidden_states[current_layer];
7     int n_embd = ctx->n_embd;
8     int n_expert = ctx->n_expert;
9
10    float sum_sq = 0.0f;
  
```

```
11     for (int d = 0; d < n_embd; ++d) {
12         sum_sq += hs[d] * hs[d];
13     }
14
15     float inv_rms =
16     1.0f / std::sqrt(sum_sq / n_embd + ctx->clg.norm_eps);
17
18     float* tl_hs_norm =
19     (float*) alloca(n_embd * sizeof(float));
20
21     for (int d = 0; d < n_embd; ++d) {
22         tl_hs_norm[d] =
23         hs[d] * inv_rms * ctx->clg.norm_w[d];
24     }
25
26     float* scores =
27     (float*) alloca(n_expert * sizeof(float));
28
29     for (int e = 0; e < n_expert; ++e) {
30         const float* gw = ctx->clg.gate_w[e];
31         float score = 0.0f;
32
33         for (int d = 0; d < n_embd; ++d) {
34             score += tl_hs_norm[d] * gw[d];
35         }
36
37         scores[e] = score;
38     }
39
40     int n_select = ctx->clg.top_k + ctx->clg.delta;
41
42     uint64_t mask = 0;
43
44     for (int k = 0; k < n_select; ++k) {
45         int best = -1;
46         float best_score = -1e38f;
47
```

```

48     for (int e = 0; e < n_expert; ++e) {
49         if (!(mask & (1ull << e)) &&
50             scores[e] > best_score) {
51
52             best_score = scores[e];
53             best = e;
54         }
55     }
56
57     if (best >= 0) {
58         mask |= (1ull << best);
59     }
60 }
61
62 int next_layer = (current_layer + 1) % ctx->n_layers;
63
64 int ids[64];
65 int n = 0;
66
67 for (int e = 0; e < n_expert && n < 64; ++e) {
68     if (mask & (1ull << e)) {
69         ids[n++] = e;
70     }
71 }
72
73 llama_moe_buffer_prefetch(
74     ctx->moe_buffer,
75     next_layer,
76     ids,
77     n
78 );
79 }
80

```

实验表明, δ 提升可显著增加 I/O 与计算重叠时间, 在 $\delta = 8$ 时达到约 93%–94% 命中率。

4.4.2 MoE-Buffer 三态管理与指针重定向

MoE-Buffer 以 expert 为单位维护三态生命周期:

- COLD: 未加载
- INFLIGHT: 加载中
- RESIDENT: 已驻留

核心机制通过 LRU + budget 控制驻留集合。

```
1 // src/llama-moe-buffer.cpp
2 void moe_stream_slice(
3 moe_buffer_context& ctx,
4 moe_managed& m,
5 int e
6 ) {
7     std::unique_lock<std::mutex> lk(ctx.mtx);
8
9     for (;;) {
10         if (m.resident[e] == ST_RESIDENT) {
11             ctx.lru.splice(ctx.lru.begin(), ctx.lru, m.lru_pos[e]);
12             ctx.hits++;
13             return;
14         }
15
16         if (m.resident[e] == ST_INFLIGHT) {
17             ctx.cv_done.wait(lk);
18             continue;
19         }
20
21         break;
22     }
23
24     m.resident[e] = ST_INFLIGHT;
25
26     while (ctx.resident_bytes + m.stride >
27            ctx.params.budget_bytes) {
28
```



```

29     if (!moe_evict_lru(ctx)) break;
30 }
31
32 ctx.resident_bytes += m.stride;
33 lk.unlock();
34
35 bool ok = moe_pread_slice(m, e, ctx.align);
36
37 lk.lock();
38
39 if (ok) {
40     m.resident[e] = ST_RESIDENT;
41     ctx.lru.push_front({&m, e});
42     m.lru_pos[e] = ctx.lru.begin();
43 } else {
44     m.resident[e] = ST_COLD;
45     ctx.resident_bytes -= m.stride;
46 }
47
48 ctx.cv_done.notify_all();
49 }
50

```

关键机制是 INFLIGHT 规避重复 I/O + LRU 维护访问局部性。

4.4.3 O_DIRECT 对齐并行读

系统使用 O_DIRECT 绕过 page cache，实现用户态直接 I/O 控制。

由于 expert slice 可能非对齐，因此采用“对齐 + bounce buffer”策略：

```

1 // src/llama-moe-buffer.cpp
2 bool moe_pread_slice(
3 moe_managed& m,
4 int e,
5 size_t align
6 ) {
7     size_t foff = m.slice_offset[e];
8     size_t stride = m.slice_size[e];
9     uint8_t* dst = m.buf + m.slice_buf_off[e];

```

```

10
11     size_t aoff = foff & ~(align - 1);
12     size_t head = foff - aoff;
13
14     size_t want =
15         (head + stride + align - 1) & ~(align - 1);
16
17     static thread_local uint8_t* bounce = nullptr;
18
19     if (!bounce) {
20         bounce =
21             (uint8_t*) aligned_alloc(align, 2 * 1024 * 1024);
22     }
23
24     ssize_t r = pread(m.fd, bounce, want, (off_t)aoff);
25
26     if (r < 0 || (size_t)r < head + stride) {
27         return false;
28     }
29
30     std::memcpy(dst, bounce + head, stride);
31     return true;
32 }
33

```

多 worker 并行读取提升 I/O 吞吐至约 2.4×。

4.4.4 热专家保护策略

系统避免高频 expert 被频繁驱逐，引入相对频率判定：

$$hot(e) = \frac{act_e}{avg_act} > \rho \quad (6)$$

实现如下：

```

1 // src/llama-moe-buffer.cpp
2 bool is_hot_expert(moe_buffer_context& ctx, int e) {
3     uint64_t total_act = ctx.total_act;
4     if (total_act == 0) return false;
5

```

```
6     float avg_act =  
7     (float) total_act / ctx.n_experts;  
8  
9     return (float)ctx.activation[e] >  
10    ctx.params.hot_ratio * avg_act;  
11 }  
12
```

驱逐时跳过 hot expert，避免稳定热点集合被破坏。

4.4.5 自适应预算与 repack 控制

系统根据 cgroup 读取 memory.max 与 memory.current，动态决定 MoE buffer budget:

- memory 足够 → budget = 0 (全驻留)
- memory 不足 → 启用 LRU + budget 限制

同时关闭 repack:

```
1 // src/llama-model.cpp  
2 if (llama_window_enabled() ||  
3 llama_moe_buffer_enabled() ||  
4 llama_flex_enabled()) {  
5     params.use_extra_bufs = false;  
6 }  
7
```

实验结果表明该策略可同时降低 RSS 并提升吞吐。

4.4.6 机制总结

MoE 流式加载模块实现了以下关键能力:

- 基于 CLG 的 expert 预测调度
- 基于 buffer 的三态生命周期管理
- 基于 O_DIRECT 的用户态 I/O 控制
- 基于 hotness 的稳定性保护
- 基于 budget 的动态内存约束

整体形成 MoE 权重从 mmap 被动访问到用户态主动调度的完整迁移路径。

4.5 KV Cache 运行时回收模块

KV Cache 运行时回收模块面向多会话、长上下文和 idle/resume 类推理场景，目标是在不改变 attention 语义的前提下，将逻辑上下文状态与物理驻留状态解耦。原生 llama.cpp 的 KV Cache 采用预分配的连续 K/V tensor，并通过 cell 元数据维护 token、position 和 sequence ownership。该设计能够保证通用推理的稳定性，但在多会话场景中，即使某个 session 暂时进入 idle 状态，其历史 KV 仍会长期占据物理页。

本模块以 block 为粒度维护 KV Cache 的运行状态：对于当前 active request 不可见、且只属于 idle session 的 KV block，系统通过 paged row index 将其从 active attention read window 中移除；随后将其 K/V 数据保存到 backing store，并对原 KV tensor 中对应的页执行 `madvise(MADV_DONTNEED)`。当 session 恢复时，系统再通过 swap-in 与 resume-aware prefetch 分步恢复历史 KV。该机制可以在不截断上下文、不压缩 KV 数值、不修改 attention kernel 语义的前提下，降低当前物理驻留页数量。

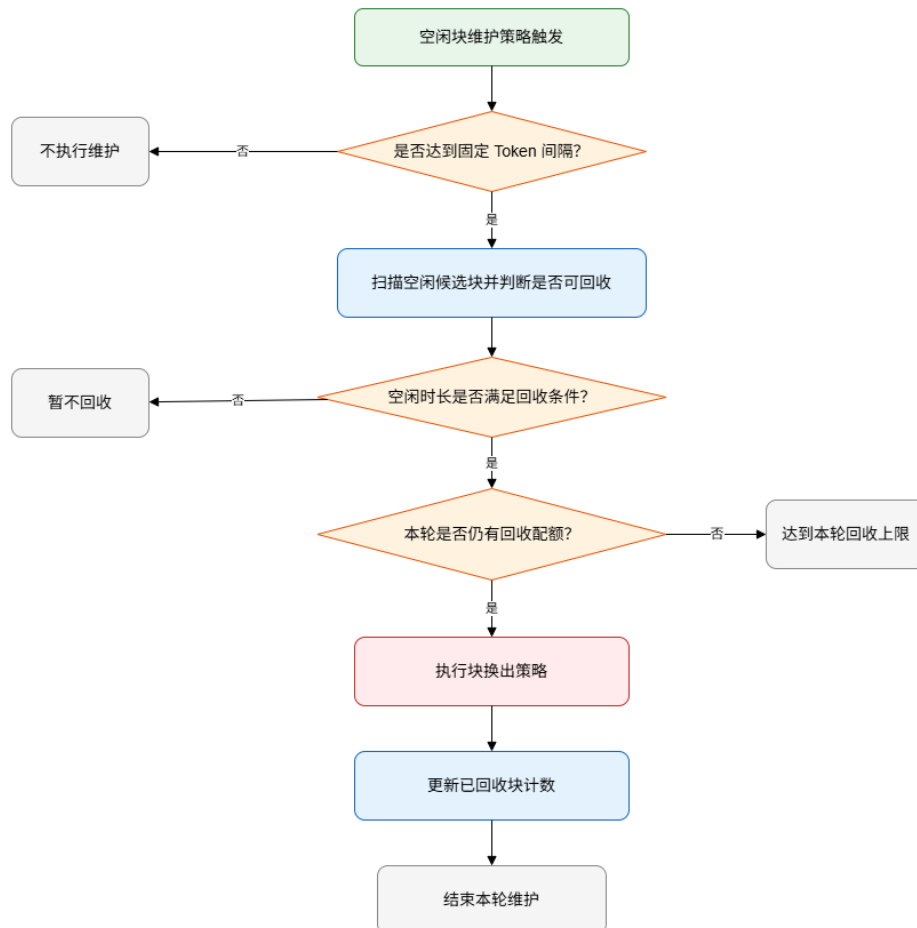


图 12: kv cache回收机制流程图

4.5.1 Lazy tail 与 lazy clear: 未使用 tail 区域的回收

在进入完整的 idle KV block reclaim 之前，系统首先实现了 lazy tail 与 lazy clear 机制，用于处理 KV buffer 中尚未被实际写入或短期内不会访问的 tail 区域。该机制保持 KV buffer 的逻辑容量不变，仅通过延迟清零和 tail madvise 避免未使用区域长期占用物理页。

需要注意的是，lazy tail 只能处理“尚未写入或逻辑上未使用”的 tail 区域。对于已经写入过、属于 idle session 的历史 KV，其内容未来 resume 时仍然需要，不能仅通过 tail reclaim 直接释放。因此，lazy tail 是完整 idle KV reclaim 的前置优化和补充机制，而不是 idle session 历史 KV 回收的替代方案。

4.5.2 Paged row index 与 idle KV 退出 active 窗口

Paged row index 是 KV Cache 回收模块的关键机制。其目标是在不重写 attention kernel 的前提下，将原本连续的 K/V 读取改造成 row-index-controlled gather。系统在 graph input 中保存 paged_row_idx，并在构图和执行阶段传递给 get_k 和 get_v。

```

1 // src/llama-graph.cpp
2 ggml_tensor* build_input_paged_row_idx(
3 llama_context* ctx,
4 ggml_context* ggml_ctx
5 ) {
6     int n_kv = llama_kv_cache_size(ctx);
7
8     ggml_tensor* row_idx =
9     ggml_new_tensor_1d(ggml_ctx, GGML_TYPE_I32, n_kv);
10
11     ggml_set_name(row_idx, "paged_row_idx");
12
13     return row_idx;
14 }
15

```

在 set_input_paged_row_idx 填充 row index 时，如果某一行所属 block 已经是 SWAPPED，系统判断 active request 是否真正需要该行。若不需要，则将该行重定向到 resident dummy row，避免 active attention 重新触碰已被 madvise 释放的物理页。

```

1 // src/llama-kv-cache.cpp
2 void set_input_paged_row_idx(...) {
3     for (uint32_t r = 0; r < n_kv; ++r) {
4         uint32_t block = r / paged_block_size;

```

```

5
6     if (paged_block_states[block] ==
7         paged_block_state::SWAPPED) {
8         bool needed_by_active =
9             logical_seq_has && unmasked_for_active;
10
11         if (needed_by_active) {
12             paged_swapped_active_visible_violation++;
13         } else if (dummy_phys != PAGED_BLOCK_INVALID) {
14             phys = dummy_phys;
15         }
16     }
17
18     data[r] = (int32_t) phys;
19 }
20 }
21

```

`paged_row_idx` 是长度为 `n_kv` 的 I32 graph input 张量，构图时创建，执行时填入当前 step 的物理 row 映射。若当前配置不支持 paged gather，则系统自动退回原始连续视图路径。

4.5.3 Swap-out 与 madvise 物理页释放

一个 KV block 只有在满足多个条件时才允许 swap-out：当前状态为 RESIDENT，属于某个 idle session，不包含 active owner，已退出 active read window，不属于 resume-pending protected seq，idle 时间达到策略阈值，未处于 defer 窗口，且本轮未超过回收配额。

当 block 被判定为可回收后，系统先逐 cell 将该 block 对应的 K/V 数据写入 backing store，然后将其状态标记为 SWAPPED，最后对原 KV buffer 对应页执行 `madvise(MADV_DONTNEED)`。

```

1 // src/llama-kv-cache.cpp
2 void paged_swap_out_block(
3     llama_kv_cache* cache,
4     uint32_t block
5 ) {
6     for (uint32_t cell = begin; cell < end; ++cell) {
7         for (auto& layer : cache->layers) {

```

```
8         ggml_backend_tensor_get(  
9         layer.k_stream[0],  
10        staging.data(),  
11        cell * row_size,  
12        row_size  
13        );  
14  
15        ggml_backend_tensor_get(  
16        layer.v_stream[0],  
17        staging.data() + kv_bytes,  
18        cell * row_size,  
19        row_size  
20        );  
21    }  
22  
23    cache->backing_store->write_cell(  
24    cell,  
25    staging.data(),  
26    staging.size(),  
27    offset  
28    );  
29  
30    cache->paged_swap_offsets[cell] = offset;  
31    cache->paged_swap_sizes[cell] = staging.size();  
32 }  
33  
34 cache->paged_block_states[physical_block] =  
35 paged_block_state::SWAPPED;  
36  
37 cache->paged_blocks_swapped_out++;  
38  
39 uint64_t advised_bytes =  
40 paged_madvise_block(cache, physical_block);  
41  
42 cache->paged_swap_madvise_bytes += advised_bytes;  
43 }  
44
```

`madvise` 只对完整落在当前 block 内部的页执行，避免误伤相邻 live block。

```

1 // src/llama-kv-cache.cpp
2 uint64_t paged_madvise_block(
3 llama_kv_cache* cache,
4 uint32_t block
5 ) {
6     uintptr_t lo =
7     (uintptr_t) cache->k_buf->data + block * block_size;
8     uintptr_t hi = lo + block_size;
9
10    size_t pg = sysconf(_SC_PAGESIZE);
11
12    uintptr_t a_start = (lo + pg - 1) & ~(pg - 1);
13    uintptr_t a_end = hi & ~(pg - 1);
14
15    if (a_end <= a_start) return 0;
16
17    size_t len = a_end - a_start;
18
19    if (madvise((void*) a_start, len, MADV_DONTNEED) == 0) {
20        return len;
21    }
22
23    return 0;
24 }
25

```

该路径体现了“先保存、再释放”的原则：只有当 backing store 写入成功后，系统才允许标记 block 为 SWAPPED 并释放物理页。

4.5.4 Resume swap-in 与 prefetch

当 session 恢复时，系统从 backing store 读回 K/V 数据，并写回原 KV tensor，使 block 状态从 SWAPPED 重新变为 RESIDENT。

```

1 // src/llama-kv-cache.cpp
2 bool paged_swap_in_block(
3 llama_kv_cache* cache,
4 uint32_t physical_block

```



```
5 ) {
6     for (uint32_t cell = begin; cell < end; ++cell) {
7         cache->backing_store->read_cell(
8             cell,
9             staging.data(),
10            staging.size(),
11            offset
12        );
13
14        for (auto& layer : cache->layers) {
15            ggml_backend_tensor_set(
16                layer.k_stream[0],
17                staging.data(),
18                cell * row_size,
19                row_size
20            );
21
22            ggml_backend_tensor_set(
23                layer.v_stream[0],
24                staging.data() + kv_bytes,
25                cell * row_size,
26                row_size
27            );
28        }
29    }
30
31    cache->paged_block_states[physical_block] =
32    paged_block_state::RESIDENT;
33
34    cache->paged_blocks_swapped_in++;
35
36    return true;
37 }
38
```

为降低 resume first-token latency，系统实现分步 prefetch。每次只恢复指定数量的 swapped blocks，从而将恢复成本分摊到 resume-pending 阶段或 active decode 间隙。

```

1 // src/llama-kv-cache.cpp
2 int32_t llama_kv_cache_prefetch_seq_step(
3 llama_kv_cache* cache,
4 uint32_t seq_id,
5 int32_t max_blocks
6 ) {
7     int32_t prefetched = 0;
8
9     for (uint32_t block : cache->seq_blocks[seq_id]) {
10         if (cache->paged_block_states[block] ==
11             paged_block_state::SWAPPED) {
12             if (!llama_kv_cache_swap_in_block(cache, block)) {
13                 break;
14             }
15
16             if (++prefetched >= max_blocks) {
17                 break;
18             }
19         }
20     }
21
22     return prefetched;
23 }
24

```

系统同时设置 prefetch protected 标记，避免刚恢复的 blocks 被重新换出；在 resume first-token 阶段使用 defer 机制，暂缓新的 idle swap-out。

4.5.5 Fast maintenance 策略

早期 aggressive idle swap 策略虽然可以释放更多页，但会在 decode 热路径中反复进行 block 分类、owner 判断和 swap-out 决策，导致 TPS 明显回退。为此，系统实现 fast maintenance 策略，通过降频、限量和 min idle 过滤控制开销。

```

1 // src/llama-kv-cache.cpp
2 void llama_kv_cache_idle_maintenance(
3 llama_kv_cache* cache
4 ) {
5     static int step_counter = 0;
6

```

```

6
7     if (++step_counter % cache->every_tokens != 0) {
8         return;
9     }
10
11     int swapped_this_step = 0;
12
13     for (uint32_t block : idle_candidates) {
14         if (cache->block_idle_steps[block] <
15             cache->min_idle_steps) {
16             continue;
17         }
18
19         if (swapped_this_step >= cache->max_blocks_per_step) {
20             break;
21         }
22
23         if (paged_swap_out_block(cache, block)) {
24             swapped_this_step++;
25         }
26     }
27 }
28

```

最终推荐配置为:

- EVERY_TOKENS = 8
- MAX_BLOCKS_PER_STEP = 8
- MIN_IDLE_STEPS = 16
- DEBUG_PROBES = 0

该配置能够避免短 idle session 被过早换出后又立即恢复, 减少 swap-out/swap-in 抖动。

4.5.6 mincore resident page 诊断

RSS 是进程级指标, 可能受模型权重、compute buffer、page cache 等因素影响。为了证明 RSS 下降来自 KV Cache resident pages, 系统使用 mincore(2) 诊断, 只对 KV tensor 范围内的虚拟地址采样, 统计 resident 页数并计算 KV resident bytes。

```
1 // src/llama-kv-cache.cpp
2 void llama_kv_cache_mincore_sample(
3 llama_kv_cache* cache
4 ) {
5     size_t pg = sysconf(_SC_PAGESIZE);
6
7     uint64_t kv_total = 0;
8     uint64_t kv_resident = 0;
9     uint64_t paged_mincore_failures = 0;
10
11     auto sample_tensor =
12     [&](ggml_tensor * t,
13     uint64_t & kind_total,
14     uint64_t & kind_resident) {
15         const uintptr_t a_start =
16         ((uintptr_t) t->data + pg - 1) & ~(pg - 1);
17
18         const uintptr_t a_end =
19         ((uintptr_t) t->data + ggml_nbytes(t))
20         & ~(pg - 1);
21
22         const size_t len = (size_t) (a_end - a_start);
23         const uint64_t n_pages = len / pg;
24
25         std::vector<unsigned char> vec(n_pages);
26
27         if (mincore((void *) a_start, len, vec.data()) != 0) {
28             paged_mincore_failures += 1;
29             return;
30         }
31
32         uint64_t res = 0;
33
34         for (uint64_t i = 0; i < n_pages; ++i) {
35             if (vec[i] & 1u) {
36                 res += 1;
```

```

37         }
38     }
39
40     kind_total += n_pages * pg;
41     kind_resident += res * pg;
42 };
43
44 for (auto& layer : cache->layers) {
45     sample_tensor(layer.k_stream[0], kv_total, kv_resident);
46     sample_tensor(layer.v_stream[0], kv_total, kv_resident);
47 }
48
49 cache->paged_mincore_kv_total_bytes = kv_total;
50 cache->paged_mincore_kv_resident_bytes = kv_resident;
51 cache->paged_mincore_failures = paged_mincore_failures;
52 }
53

```

`mincore` 将测量对象限制在 KV tensor 页，而不是整个进程 RSS。因此，它能够直接证明 `madvise` 后 KV resident pages 是否下降，以及 swapped pages 是否被读路径重新 fault-in。

4.6 正确性保证

本项目在内存优化过程中必须确保推理结果与原生 `mmap` 模式一致，不因预取、回收、换出或恢复操作引入语义变化。为此，系统从以下几个层面保证正确性。

4.6.1 只读 `mmap` 与数据不变性

权重页通过 `mmap` 以只读方式映射，系统不对页内容做任何修改。`MADV_DONTNEED` 丢弃的只是物理页副本，并不影响文件映射本身。当推理线程再次访问该页时，内核会从文件重新读取原始数据。因此，无论页被回收多少次，每次读回的内容都保持一致。

4.6.2 跨层共享页保护

在模型加载后的索引构建阶段，系统按地址排序并对齐各层权重区间。若不同层的两个区间落在同一物理页内，系统将该页标记为不可回收。这样可以避免因回收某一层权重页而间接影响另一层仍需访问的数据。

4.6.3 回收前二次检查

回收任务入队后，worker 线程并不立即执行回收。在真正调用 `madvise(MADV_DONTNEED)` 之前，线程会再次检查目标层当前是否仍处于保护窗口之外。该机制用于处理窗口推进与后台回收之间的并发竞态，避免回收即将被访问的层。

4.6.4 窗口状态的原子更新

窗口控制器中的当前层号、保护窗口范围等状态可能被推理主线程和 worker 线程同时访问。系统使用互斥锁保护这些状态的读写操作，确保任一时刻只有一个线程能够修改窗口状态，避免状态不一致导致错误回收或重复预取。

4.6.5 KV Cache 的先保存、再释放原则

对于 KV Cache，系统只对 idle-owned 且 active-invisible 的 block 执行 swap-out。执行物理页释放前，系统必须先将对应 K/V 数据写入 backing store，并记录 offset 与 size。只有 backing store 写入成功后，block 才能被标记为 SWAPPED 并执行 `madvise`。当 session resume 时，系统再从 backing store 恢复历史 KV，从而保证上下文语义不被破坏。

4.7 配置参数

本系统支持通过环境变量进行参数配置。主要配置项如表 6 和表 7 所示。

表 6: Window 与权重流式模块配置参数

环境变量	说明	默认值
LLAMA_LAZY_V2	启用 Window 框架	0
LLAMA_LAZY_WINDOW_SIZE	窗口层数，即 $A + B + 1$	12
LLAMA_LAZY_PREFETCH_AHEAD	预取提前层数 A	2
LLAMA_LAZY_WORKER_THREADS	后台 worker 线程数	1
LLAMA_LAZY_DONTNEED	启用 MADV_DONTNEED 回收	1
LLAMA_LAZY_MEMORY_LIMIT	内存上限，单位 GB，0 表示不限	0
LLAMA_LAZY_AUTO_TUNE	启用自适应调优	1
LLAMA_LAZY_CLG	启用 CLG 跨层专家预测	0
LLAMA_LAZY_CLG_DELTA	CLG 过预测余量 δ	按需配置
LLAMA_LAZY_MOE_BUFFER	启用 MoE-Buffer 专家流式机制	0

环境变量	说明	默认值
LLAMA_LAZY_MOE_BUFFER_WORKERS	MoE-Buffer 并行读取线程数	按需配置
LLAMA_FLEX	启用 Flex 稠密层匿名环形缓冲	0
LLAMA_FLEX_AUTO	启用 Flex 自适应 ring 大小	1

表 7: KV Cache 回收模块配置参数

环境变量	说明	推荐值
LLAMA_KV_LAZY_TAIL	启用未使用 tail 区域 lazy reclaim，降低尚未写入或短期内不会访问的 KV tail 物理驻留。	1
LLAMA_KV_LAZY_CLEAR	启用 lazy clear，避免在不必要时立即清理完整 KV 区域。	1
LLAMA_KV_PAGED	启用 block-level paged KV 管理机制。	1
LLAMA_KV_PAGED_INGRAPH	在 attention graph 中接入 paged_row_idx。	1
LLAMA_KV_PAGED_GATHER_NONIDENTITY	启用 non-identity row gather，使 idle-only / active-invisible rows 能够重定向到 dummy resident row。	1
LLAMA_KV_PAGED_SWAP	启用 KV block swap-out / swap-in 基础路径。	1
LLAMA_KV_PAGED_IDLE_SWAP	启用 idle KV block 回收策略。	1
LLAMA_KV_PAGED_IDLE_SWAP_MADVISE	swap-out 后对原 KV tensor 页执行 <code>madvise(MADV_DONTNEED)</code> ，释放对应物理页。	1
LLAMA_KV_PAGED_PREFETCH_DURING_ACTIVE	在 active decode 间隙执行 resume-aware prefetch。	1
LLAMA_KV_PAGED_PREFETCH_AUTO_EVERY_TOKENS	自动 prefetch 调度的 token 间隔。	2
LLAMA_KV_PAGED_PREFETCH_AUTO_BLOCKS_PER_STEP	每次 prefetch 最多恢复的 block 数。	2

环境变量	说明	推荐值
LLAMA_KV_PAGED_PREFETCH_AUTO_ SAFETY_TOKENS	自动 prefetch 起点的安全余量。	0
LLAMA_KV_PAGED_PREFETCH_ FINAL_SYNC_BLOCKS	resume 首 token 前 final sync 最多同 步恢复的 block 数。	12
LLAMA_KV_PAGED_DEFER_SWAPOUT_ ON_RESUME	resume 阶段延迟新的 idle swap-out, 避免恢复后立即再次换出。	1
LLAMA_KV_PAGED_IDLE_SWAP_ DEBUG_PROBES	debug probe 开关, 正式测试中关闭以 降低热路径开销。	0
LLAMA_KV_PAGED_IDLE_SWAP_ EVERY_TOKENS	每隔多少 token 执行一次 idle mainte- nance。	8
LLAMA_KV_PAGED_IDLE_SWAP_MAX_ BLOCKS_PER_STEP	每次 maintenance 最多 swap-out 的 block 数。	8
LLAMA_KV_PAGED_IDLE_SWAP_MIN_ IDLE_STEPS	block 至少 idle 多少步后才允许回收, 用于降低短 idle 抖动。	16
LLAMA_KV_PAGED_MINCORE	启用 mincore resident page 诊断, 用 于区分 KV resident drop 与进程级 RSS 波动。	按需开启

4.8 与 llama.cpp 原有功能的兼容性

本项目在设计上遵循最小侵入原则, 所有改动均通过新增独立模块和回调机制实现, 不修改 llama.cpp 原有核心计算流程和主要数据结构。

在 mmap 兼容性方面, 本方案在原有 mmap 基础上进行扩展, 不改变模型文件映射方式。若用户通过 `--no-mmap` 禁用 mmap, Window 方案会自动禁用并回退至原生加载方式。

在量化格式方面, 本方案与 GGUF 量化格式兼容, 不改变权重数据的存储格式和加载方式, 用户已有的量化模型无需转换即可使用。

在计算后端方面, 本方案通过节点回调机制与 GGML 计算图集成, 不与具体后端强绑定。当前主要验证 CPU 后端路径; 对于 GPU backend, 尤其是 KV Cache 位于 device memory 或 backend buffer 的场景, 仍需进一步设计 device-host transfer、DMA、统一内存或显存页管理等适配路径。

需要注意的是, 使用本方案时应禁用 `mlock` 功能, 防止锁定物理页干扰回收机制; 同时应禁用 `extra_bufs` 权重重排功能, 因为重排会分配独立内存并拷贝权重数据, 破

坏零拷贝与可控回收前提。

4.9 主要改动文件清单

表 8: 主要改动文件清单

文件	改动内容
src/llama-window.{h,cpp}	Window 控制器、CLG 预测、节点完成回调入口。
src/llama-moe-buffer.{h,cpp}	MoE 专家流式、O_DIRECT 对齐读、三态状态机、驱逐策略、权重流回调。
src/llama-flex.{h,cpp}	Dense 层匿名环形缓冲、层粒度 LRU 替换、自适应 ring 管理。
ggml/include/ggml-cpu.h	新增 node_callback 与 weight_stream_callback 接口声明。
ggml/src/ggml-cpu/ggml-cpu.c	在图节点完成后与算子执行前增加回调触发点。
src/llama-model.cpp	模型加载时建立层索引，初始化 Window、MoE-Buffer、Flex，并在必要时关闭 repack。
src/llama-context.cpp	在 decode 前后安装与卸载图执行回调。
src/llama-kv-cache.{h,cpp}	KV block 状态管理、swap-out、swap-in、madvise、mincore、idle maintenance。
src/llama-graph.{h,cpp}	在 attention graph 中接入 paged_row_idx。
examples/kv-trace-replay/	多 session、idle/resume trace replay driver。

4.10 核心函数说明

表 9: 核心函数说明

函数名	所在文件	作用
llama_window_create	llama-window.cpp	创建窗口控制器，建立层索引，启动 worker 线程。
llama_window_advance_locked	llama-window.cpp	推进滑动窗口，派发预取和回收任务。

函数名	所在文件	作用
<code>llama_window_is_protected</code>	<code>llama-window.cpp</code>	判断某层是否处于保护窗口内。
<code>llama_window_populate</code>	<code>llama-window.cpp</code>	执行预取操作，将指定范围 <code>fault</code> 进物理内存。
<code>llama_window_worker</code>	<code>llama-window.cpp</code>	<code>worker</code> 线程主循环，处理预取与回收任务。
<code>llama_window_node_done</code>	<code>llama-window.cpp</code>	节点完成回调处理函数，解析层号并推进窗口。
<code>llama_window_clg_predict_and_schedule</code>	<code>llama-window.cpp</code>	执行 CLG 跨层预测，计算下一层专家打分并派发预取请求。
<code>llama_moe_buffer_stream_callback</code>	<code>llama-moe-buffer.cpp</code>	处理 MoE 专家权重重指向，并执行同步兜底。
<code>moe_stream_slice</code>	<code>llama-moe-buffer.cpp</code>	加载专家切片，维护 COLD/IN-FLIGHT/RESIDENT 三态状态机。
<code>moe_pread_slice</code>	<code>llama-moe-buffer.cpp</code>	通过 <code>O_DIRECT</code> 对齐读从文件读取专家切片到匿名缓冲。
<code>is_hot_expert</code>	<code>llama-moe-buffer.cpp</code>	基于相对频率判断专家是否为热专家。
<code>flex_acquire_slot</code>	<code>llama-flex.cpp</code>	为 Dense 层分配匿名环形缓冲槽位，必要时驱逐旧层。
<code>llama_kv_cache_swap_out_block</code>	<code>llama-kv-cache.cpp</code>	将 KV block 写入 backing store，并执行 <code>madvise</code> 释放物理页。
<code>llama_kv_cache_swap_in_block</code>	<code>llama-kv-cache.cpp</code>	从 backing store 读回 KV block 数据。
<code>llama_kv_cache_prefetch_seq_step</code>	<code>llama-kv-cache.cpp</code>	分步 prefetch，每次恢复指定数量的 swapped blocks。
<code>llama_kv_cache_mincore_sample</code>	<code>llama-kv-cache.cpp</code>	使用 <code>mincore</code> 诊断 KV tensor resident 页。

5 系统测试

本节介绍本项目在初赛阶段的系统测试结果，包括测试环境、内存占用测试、推理性能测试、正确性验证、参数敏感性分析、KV Cache 回收模块测试结果、测试结论汇总以及现阶段存在的问题。

5.1 测试环境

本项目的测试环境参数如表 10 所示。所有实验均在同一物理机、同一操作系统和同一存储环境下完成，以保证不同方案之间的可比性。对比环境包括 llama.cpp 原生 mmap 路径、Flex/Window 权重流式路径、MoE-Buffer 专家缓冲路径、CLG 异步预取路径以及 Paged-KV 运行时回收路径。各优化功能均通过环境变量启用或关闭，无需切换系统环境。

表 10: 测试环境参数

环境指标	环境参数
Linux 发行版	Ubuntu 22.04
Linux 内核	5.15.0+
机器模式	物理机
内存大小	23GB
处理器	x86-64, 8 核
存储	SSD, 顺序读速约 279MB/s
Dense 测试模型	Llama-3-8B-Instruct-Q4_K_M.gguf
Dense 模型大小	约 4.58GB
MoE 测试模型	Qwen1.5-MoE-A2.7B-20-experts-SFT-trained.Q4_K_M.gguf
MoE 模型结构	24 个 MoE layer, 每层 20 个 expert, top-4 激活
内存限制方式	cgroup v2 (memory.max)
测试工具	llama-bench、perplexity、KV trace replay 示例程序

本项目测试主要关注三个维度：第一，系统是否能够显著降低物理内存占用；第二，系统是否能够在内存下降的同时维持可接受的推理吞吐；第三，系统是否能够保证推理结果与原生路径一致，不改变模型计算语义。

5.2 权重管理模块测试结果

为系统性评估权重管理模块在不同模型结构与内存约束下的表现，实验从三个层面展开：(1) 内存占用控制能力；(2) 权重流式与缓存机制有效性；(3) 不同内存预算与并行策略下的性能变化。

5.2.1 内存占用与repack优化收益

首先评估系统在稳态下的内存占用表现。内存通过 `/proc/<pid>/status` 中 RSS 字段获取，结果如表 11 所示。

表 11: 权重管理模块内存占用与repack优化效果

配置	RSS (GB)	吞吐 (tok/s)	说明
Dense native (含 repack)	7.77	8.15	mmap + repack副本
Dense (关闭 repack)	4.56	8.60	消除匿名权重副本
MoE native (含 repack)	5.73	19.26	expert全量驻留
MoE (关闭 repack)	3.60	19.58	缓冲区直连权重文件

结果表明，`repack` 机制在当前权重管理体系中引入了额外的中间布局与匿名缓冲副本，其本质是为了兼容传统 `mmap` 权重访问路径而进行的临时数据重排。然而，在本系统的设计中，Dense 路径已经通过 Window + Flex Buffer 实现了分层窗口化驻留，MoE 路径则通过 expert-level MoE-Buffer 直接维护受控权重集合，使得原有 `repack` 所依赖的中间缓冲结构变为冗余。

5.2.2 Dense模型路径：滑动窗口权重驻留机制

Dense 模型采用 Window + Flex Buffer 机制，仅维护局部层窗口，其余层按需加载。在不同内存约束下，其表现如表 12 所示。

表 12: Dense模型在不同内存cap下的表现

cap (MB)	RSS (MB)	tg (tok/s)	说明
native	7770	8.15	全量page fault驻留
window	4560	8.60	仅窗口层常驻
3072-128	=cap	0.16-0.18	IO受限，完全I/O bound

可以观察到，在无内存约束场景下，Dense 模型通过 Window 机制将权重驻留范围从“全模型驻留”压缩为“局部窗口驻留”，显著降低 RSS。由于层访问具有连续性，预取线程能够提前加载后续层，使计算线程基本不受 I/O 影响，因此在 RSS 下降约 44% 的同时，吞吐由 8.15 tok/s 略升至 8.60 tok/s。

当内存 cap 进一步低于模型工作集规模时，Flex Buffer 难以维持有效层窗口，系统退化为按需加载模式。由于 Dense 模型每个 token 都需要遍历全部层权重，窗口外权重会被反复加载与回收，执行瓶颈转向 page fault 与磁盘 I/O，吞吐下降至 0.16–0.18 tok/s。

该结果说明，Window + Flex Buffer 能够在内存相对充足时有效压缩 RSS 并隐藏 I/O 延迟；但在极端压缩下，Dense 模型缺乏稀疏性，性能下界仍由存储带宽决定。

5.2.3 MoE模型路径：缓冲区工作边界与buffer有效性

MoE模型采用 expert-level MoE-Buffer，并结合CLG预测进行预取。

(1) 缓冲区边界分析 不同内存budget下系统工作状态如表 13 所示。

表 13: MoE-Buffer拐点

budget	evictions	read/token	tg (tok/s)	状态
768	7392	598	1.97	严重抖动
1536	2710	272	2.23	过渡状态
2048	618	132	6.22	收敛
2432	0	~0	11.57	拐点
2560	0	~0	9.43	稳态

结果显示，MoE-Buffer 在不同 budget 下存在明显的工作集边界：当 budget 较低时，缓冲区无法容纳反复访问的高频 expert，系统需要在 COLD / INFLIGHT / RESIDENT 状态之间频繁切换，并不断驱逐已有 expert，此时系统处于抖动或过渡状态，吞吐仅为 1.97–2.23 tok/s。

当 budget 增至 2048MB 后，缓冲区开始覆盖主要活跃 expert，吞吐提升至 6.22 tok/s。当 budget 达到 2432MB 时，evictions 降为 0，read/token 接近 0，说明缓冲区已覆盖当前负载下的有效 expert 工作集，系统进入稳定命中状态。此时后台预取主要用于冷启动或少量路由变化，不再产生持续读盘。2560MB 下吞吐略低于 2432MB，主要可归因于运行噪声、线程调度或缓存状态差异。

该结果说明，MoE-Buffer 的性能关键在于是否覆盖活跃 expert 工作集。低于该边界时，系统会产生缓存抖动和 I/O 放大；达到边界后，权重访问主要由缓冲区命中完

成，吞吐显著提升。

(2) CLG预测与预取效果 CLG预测中的过预测参数 δ 对性能影响如表 14 所示。

表 14: CLG预测与预取效果

δ	RSS (MB)	命中率	tg (tok/s)	说明
0	2411	94%	12.52	基线
4	2579	93%	16.48	IO-Compute overlap增强
8	2634	93%	17.27	性能最优区间

由测试结果可知，当 $\delta = 0$ 时，CLG 仅按照基线预测范围进行预取，虽然命中率较高，但部分 expert 的加载时机仍可能接近实际访问时刻，导致计算线程需要等待 INFLIGHT 状态完成。当 δ 增至 4 或 8 后，系统对候选 expert 进行适度过预测，增加了 I/O 与计算之间的时间间隔，从而提高了 overlap 程度，减少同步等待。

此时，RSS 仅从 2411MB 增至 2634MB，说明过预测带来的额外驻留开销较小，仍处于 MoE-Buffer 可控范围内。因此，适度增大 δ 能够以有限内存增长换取更充分的预取提前量，是 MoE 权重流式管理中的有效性能调节参数。

(3) 并行IO扩展性 MoE-Buffer支持多worker并行预取，其扩展性如表 15 所示。

表 15: MoE多worker IO加速

budget	1 worker	4 worker	加速比
512	1.58	2.87	1.8×
768	1.97	3.75	1.9×
1536	2.23	5.42	2.4×

结果显示，在较低 budget 下，MoE-Buffer 处于明显的 I/O 主导区间，此时单个 worker 难以及时完成多个 expert 的异步加载，计算线程容易等待 INFLIGHT 状态结束。引入 4 个 worker 后，系统能够并行执行多个 pread 任务，使不同 expert 的加载过程在锁外并发完成，从而提高 I/O 吞吐并减少同步等待。

同时可发现，随着 budget 从 512MB 增至 1536MB，4 worker 相比 1 worker 分别获得 1.8×、1.9× 和 2.4× 加速。原因在于缓冲区增大能够保留更多活跃 expert，预取任务的有效性提高，因此并行 I/O 的收益进一步放大。

(4) 数值正确性验证 由于 MoE-Buffer 改变了 expert 权重的物理访问路径，并通过指针重定向使 GEMM 直接读取匿名缓冲区中的 expert slice，因此需要验证该机制不会引入数值偏差。本系统在 WikiText-2 数据集上使用 llama-perplexity 对 MoE 模型进行正确性测试，并将 mmap 基线与 MoE-Buffer + CLG 异步流式路径进行逐 chunk PPL 对比，结果如表 16 所示。

表 16: MoE-Buffer + CLG 数值正确性验证

Chunk	mmap 基线 PPL	MoE-Buffer + CLG PPL	是否一致
1	47948.2078	47948.2078	一致
2	80646.3078	80646.3078	一致
3	95059.6850	95059.6850	一致
4	104976.4199	104976.4199	一致

结果显示，MoE-Buffer + CLG 路径在多个 chunk 上得到的 PPL 与 mmap 基线完全一致。这说明 expert slice 的注册、异步加载、状态切换以及 GEMM 前的同步兜底机制均未改变权重内容或计算顺序，指针重定向仅改变权重的物理驻留位置，不改变模型的数值语义。因此，MoE-Buffer 在降低 RSS 和控制 expert 驻留集合的同时，能够保持与原始 mmap 路径一致的推理正确性。

5.2.4 极端内存约束鲁棒性

在 cgroup 限制下（Dense: 3GB / MoE: 2GB），系统表现如表 17 所示。

表 17: 极端内存约束下系统鲁棒性

模型	RSS	tg (tok/s)	结果
Dense native	—	—	OOM
Dense Window	4.26GB	0.40	IO-bound运行
MoE-Buffer	1.90GB	2.57	稳定运行

结果表明，在极端内存约束下，权重管理模块能够将 native 路径的 OOM 失败转化为可运行状态。native 依赖 mmap 被动调页，并同时受到权重页、KV Cache、激活值和 repack 副本的内存压力影响；当 cgroup 上限低于完整工作集规模时，系统无法维持必要驻留页，最终触发 OOM。

Dense Window 在 3GB 限制下可以正常运行，说明滑动窗口驻留与层级回收机制能够避免全量权重同时驻留。但由于 Dense 模型每个 token 仍需访问全部层权重，窗口外层会被反复加载和回收，执行瓶颈转向磁盘 I/O，因此吞吐降至 0.40 tok/s。

MoE-Buffer 在 2GB 限制下将 RSS 控制在 1.90GB，并保持 2.57 tok/s。其原因在于 MoE 模型具有专家稀疏激活特征，系统只需按需维护少量 active expert，并通过 COLD / INFLIGHT / RESIDENT 状态机和 CLG 预测控制专家驻留集合。

总体来看，该实验说明显式权重管理提升了内存受限场景下的鲁棒性；Dense 路径主要提供可运行性，MoE 路径则能进一步利用稀疏性获得更稳定的受限内存性能。

5.3 KV Cache 管理模块测试结果

KV Cache 回收模块主要在多会话、长上下文和 idle/resume 场景下验证。测试使用 ShareGPT-backed synthetic trace replay workload，关注 process RSS、active TPS、decode time、resume first-token latency 以及 mincore resident page 诊断结果。

5.3.1 Fast maintenance 消融结果

在 ctx4096 long-idle trace 上，系统对 default、debug off、every4、every8、budget、min_idle 等配置进行对比。最终推荐 V5 配置为：

- EVERY_TOKENS = 8
- MAX_BLOCKS_PER_STEP = 8
- MIN_IDLE_STEPS = 16
- DEBUG_PROBES = 0

三次运行取中位数的结果如表 18 所示。

表 18: Fast maintenance 消融结果

配置	RSS drop	KV drop	capacity	Active delta	TPS	Decode time delta
V4 every8 + budget8 + debug off	533.133 MiB	52.064%		-7.194%		+7.751%
V5 every8 + budget8 + idle16 + debug off	611.883 MiB	59.754%		-3.007%		+3.100%

V5 相比 V4 在 RSS drop 更高的同时，TPS 回退更低。其原因是 MIN_IDLE_STEPS=16 避免了短 idle block 被过早换出，从而降低了 swap-out/swap-in 抖动。

5.3.2 ctx4096 ShareGPT-backed trace replay 结果

在 ctx4096、8 session、ShareGPT-backed synthetic trace 下，V5 配置的三次运行中位数结果如表 19 所示。

表 19: ctx4096 ShareGPT-backed trace replay 结果

指标	Baseline	V5	变化
Process RSS	9333632 KiB	8707064 KiB	-611.883 MiB
Active TPS	10.875037	10.548073	-3.007%
Active decode time	77976.745 ms	80393.828 ms	+3.100%
Total RSS drop	—	—	6.713%
RSS drop / theoretical KV capacity	—	—	59.754%
Resume first-token weighted avg	100.251 ms	101.493 ms	+1.242 ms

在 ctx4096 场景下，系统能够回收约 612 MiB process RSS，active TPS 回退约 3%，resume first-token latency 增量约 1.2 ms。该结果说明，在中等上下文长度的多会话 idle/resume 场景中，KV Cache 回收模块能够获得可观内存收益，同时将性能回退控制在较低水平。

5.3.3 ctx8192 ShareGPT-backed trace replay 结果

在 ctx8192 场景下，KV Cache 理论容量更大，因此 idle KV reclaim 的收益更明显。V5 配置三次运行中位数结果如表 20 所示。

表 20: ctx8192 ShareGPT-backed trace replay 结果

指标	Baseline	V5	变化
Process RSS	10364540 KiB	8706484 KiB	-1619.195 MiB
Active TPS	10.846081	10.736931	-1.006%
Active decode time	78184.923 ms	78979.742 ms	+1.017%
Total wall time	96365.403 ms	97296.760 ms	+0.966%
Total RSS drop	—	—	15.997%
RSS drop / theoretical KV capacity	—	—	79.062%
Resume first-token weighted avg	103.225 ms	100.830 ms	-2.395 ms

ctx8192 下 process RSS 下降约 1.58 GiB, active TPS 仅下降约 1%。这说明随着上下文长度增加, KV Cache 在总内存中的占比提高, block-level idle reclaim 的收益更加明显。

5.3.4 mincore resident page 诊断结果

为证明 RSS drop 主要来自 KV resident page drop, 系统使用 LLAMA_KV_PAGED_MINCORE=1 进行诊断。ctx4096 结果如表 21 所示。

表 21: ctx4096 mincore resident page 诊断结果

指标	M0 baseline-like	V5	变化
KV total	1023.75 MiB	1023.75 MiB	0
KV resident	1023.75 MiB	426.0 MiB	-597.75 MiB
KV resident ratio	100.0%	41.61%	-58.39%
Process RSS drop	—	—	约 611.746 MiB

ctx8192 结果如表 22 所示。

表 22: ctx8192 mincore resident page 诊断结果

指标	M0 baseline-like	V5	变化
KV total	2047.75 MiB	2047.75 MiB	0
KV resident	2047.75 MiB	426.0 MiB	-1621.75 MiB
KV resident ratio	100.0%	20.80%	-79.20%
Process RSS drop	—	—	约 1619.168 MiB

KV resident drop 与 process RSS drop 在数量级上高度一致，说明最终 RSS 下降主要来自 KV Cache 物理驻留页的真实减少，而不是进程其他内存波动。

5.4 KV Cache 与权重侧优化的叠加效果

除单独评估权重管理模块外，本文进一步测试了 KV Cache reclaim 与权重侧 flex streaming 的组合效果。测试结果如表 23 所示。

表 23: KV Cache reclaim 与 Dense flex streaming 的叠加效果

配置	RSS (KB)	active TPS	KV swap-out	说明
default	8,683,152	14.593	0	原始路径
flex auto	5,134,680	14.749	0	权重侧降 RSS
KV-only	8,250,536	14.143	23	KV 侧单独生效
KV + flex auto	4,703,812	14.171	23	两类优化叠加

结果表明，flex auto 与 KV Cache reclaim 作用于不同模块：前者控制模型权重的物理页驻留，后者释放 idle KV blocks 对应的运行时缓存页。因此两者收益可以叠加。与 default 相比，flex auto 单独使 RSS 降低约 3.38GiB；KV-only 额外降低约 422MiB；二者组合后 RSS 降至 4,703,812KB，总体下降约 3.80GiB，降幅约 45.83%。与此同时，active TPS 仅从 14.593 降至 14.171，说明该组合在显著降低内存占用的同时仍保持了接近原始路径的推理吞吐。

6 现阶段存在的问题

尽管本系统已经完成核心功能验证，但当前仍存在若干限制和后续优化空间。

6.1 受限内存场景下的性能瓶颈

在 cgroup 限制 3GB 的极端内存受限场景下，Window 方案能够避免 OOM，使推理得以继续；但吞吐量从 7.54 tokens/s 降至 0.40 tokens/s，下降幅度超过 90%。其根本原因在于模型大小为 4.58GB，远超可用物理内存，每次窗口滑动都需要从磁盘读取新的层，而 SSD 读速远低于内存带宽，导致推理过程被 I/O 主导。

可能的改进方向包括：增大窗口大小以摊薄 I/O 开销，使用更高速的存储设备，例如 NVMe SSD，或者引入更细粒度的 block-level 权重流式与学习型预取策略。但对于内存远小于模型大小的 Dense 模型场景，性能瓶颈具有结构性，单纯依赖 mmap 路线难以彻底解决。

6.2 跨层共享页的识别

在层索引构建阶段，若不同层的权重落在同一物理页上，系统会将该页标记为不可回收。该设计保证了正确性，但当共享页较多时，实际可回收内存会减少，从而影响内存优化效果。目前尚未在更大规模模型上系统评估共享页比例及其对回收收益的影响。

6.3 当前主要验证 CPU KV path

本项目主要在 CPU KV path 下验证。GPU backend 下，KV tensor 可能位于 device memory 或 backend buffer 中，涉及 device-host transfer、DMA、统一内存或显存页管理等问题，不能直接套用 CPU 侧 madvise 语义。因此，GPU backend 需要针对设备内存和主机内存之间的数据迁移重新设计 backing store 与恢复路径。

6.4 prefetch 仍是 driver 协作式

当前 prefetch 是 driver 在 resume-pending 和 active decode 间隙调用 prefetch_seq_step 实现的分步恢复，不是完全独立的后台线程。后续可进一步实现真正的异步 prefetch worker，并结合内存压力信号、I/O 带宽和 session deadline 自动决定 prefetch 频率与恢复块数。

7 大语言模型使用说明

项目开发过程中，我们合理使用了大语言模型作为辅助工具，具体使用方式如下：

1. **文献检索与资料整理：**借助开源大语言模型（如 DeepSeek V4 等）对相关研究论文进行语义检索与摘要生成，辅助快速定位技术路线和关键参考文献，提高调研效率。

2. **开源框架梳理与分析：**利用 LLM 对 llama.cpp、vLLM、SGLang 等开源推理框架的代码仓库结构、文档说明和实现机制进行快速梳理与对比分析，辅助理解各框架的权重管理、KV Cache 组织方式以及相关优化技术，为系统设计与方案选型提供参考。
3. **代码开发辅助：**在系统实现过程中，使用 LLM 辅助生成代码框架、编写测试脚本、调试错误及优化代码结构，所有生成代码均经人工审阅、修改和验证，确保符合项目需求和正确性。

需要特别说明的是，所有技术方案设计、核心算法实现、实验设计、数据分析与结论均完全由本项目成员独立完成，LLM 仅作为辅助工具使用，项目最终成果的完整性与真实性由本项目成员全权负责。

参考文献

- [1] Zixuan Zhou, Xuefei Ning, Ke Hong, et al., A Survey on Efficient Inference for Large Language Models, arXiv preprint, 2024.
- [2] Hongchao Du, Shangyu Wu, Arina Kharlamova, et al., FlexInfer: Breaking Memory Constraint via Flexible and Efficient Offloading for On-Device LLM Inference, arXiv preprint, 2024.
- [3] Keivan Alizadeh, Iman Mirzadeh, Dmitry Belenko, et al., LLM in a flash: Efficient Large Language Model Inference with Limited Memory, arXiv preprint, 2024.
- [4] Zhenliang Xue, Yixin Song, Zeyu Mi, et al., PowerInfer-2: Fast Large Language Model Inference on a Smartphone, arXiv preprint, 2024.
- [5] Zhiyuan Fang, Zicong Hong, Yuegui Huang, et al., Fate: Fast Edge Inference of Mixture-of-Experts Models via Cross-Layer Gate, arXiv preprint, 2024.
- [6] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, et al., Efficient Memory Management for Large Language Model Serving with PagedAttention, in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023, pp. 611–626.
- [7] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, et al., SGLang: Efficient Execution of Structured Language Model Programs, arXiv preprint, 2024.
- [8] Yuhang Li, Rong Gu, Chengying Huan, et al., HotPrefix: Hotness-Aware KV Cache Scheduling for Efficient Prefix Sharing in LLM Inference Systems, arXiv preprint, 2025.

- [9] Krishna Teja Chitty-Venkata, Jie Ye, Xian-He Sun, et al., PagedEviction: Structured Block-wise KV Cache Pruning for Efficient Large Language Model Inference, arXiv preprint, 2024.
- [10] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, et al., vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention, in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2025.